

Minería de Datos - Libro 1 - Clase 1 a 10

Sitio: [Centro Politécnico Superior Malvinas Argentinas](#)

Curso: Minería de Datos (2026)

Libro: Minería de Datos - Libro 1 - Clase 1 a 10

Imprimido por: Sandra Vargas

Día: martes, 9 de junio de 2026, 15:41

Descripción



Este recurso contiene el material escrito del curso y acompañará el desarrollo de la asignatura a lo largo de todo el recorrido. En él se presentan los conceptos fundamentales de Minería de Datos junto con ejemplos y actividades prácticas que permitirán aplicar y consolidar los contenidos trabajados en clase.

Tabla de contenidos

1. Introduccion a la Minería de Datos
2. Tipos, Limpieza y Transformacion
3. EDA, Analisis Multivariado
4. Visualizacion de Datos.html
5. Limpieza y Transformacion Avanzada
6. Clustering
7. Repaso

8. Aprendizaje Supervisado, Regresión

9. Árboles de Decisión

10. Minería de Texto - TensorFlow

1. Introducción a la Minería de Datos

Clase No 1: Introducción a la Minería de Datos y Python para Análisis de Datos

1. Presentación del Curso

¡Bienvenidos al bloque de Minería de Datos!

Mi nombre es Juan Paulo Zarza, y seré su guía a lo largo de este recorrido de 10 clases del bloque.

En esta primera clase, vamos a introducir los conceptos fundamentales, explorando cómo la minería de datos transforma grandes volúmenes de información en conocimiento útil. También nos familiarizaremos con Python y sus bibliotecas para análisis de datos. Vamos a revisar los marcos procedimentales que disponemos para ordenar nuestro trabajo

Este curso seguirá un enfoque teórico-práctico, combinando la comprensión conceptual con la aplicación real a través de código en Python.

2. Introducción a la Minería de Datos

2.1. ¿Qué es la Minería de Datos?

La Minería de Datos es una disciplina que permite descubrir patrones, tendencias y correlaciones en grandes volúmenes de datos. Se apoya en técnicas de diversas áreas como:

- Estadística: cálculo de probabilidades, distribuciones, inferencias.
- Aprendizaje Automático (Machine Learning): algoritmos supervisados y no supervisados.
- Bases de Datos: recuperación, almacenamiento y optimización de consultas.
- Inteligencia Artificial: reconocimiento de patrones, redes neuronales.

En la práctica, la minería de datos se usa para predecir tendencias, optimizar procesos y mejorar la toma de decisiones en diversos sectores.

Ejemplo Práctico

Imaginemos que una cadena de supermercados quiere mejorar su estrategia de ventas. Con minería de datos puede:

- Analizar patrones de compra para hacer ofertas personalizadas.
- Detectar productos frecuentemente comprados juntos (reglas de asociación).
- Usar clustering para segmentar clientes según su comportamiento de compra.
- Implementar modelos predictivos para anticipar la demanda de productos.

Este enfoque permite transformar datos en decisiones estratégicas, lo que da a las empresas una ventaja competitiva.

2.2. Diferencias entre Datos, Información y Conocimiento

Antes de profundizar en minería de datos, es importante entender la diferencia entre dato, información y conocimiento:

Concepto	Definición	Ejemplo

Dato	Unidad mínima de información, sin contexto.	"25°C"
Información	Dato procesado con significado.	"La temperatura actual es de 25°C"
Conocimiento	Interpretación de la información para la toma de decisiones.	"Dado que hace calor, la venta de bebidas frías aumentará"

Ejemplo en un contexto empresarial

Un negocio de ventas en línea recopila miles de registros diarios.

- Dato: "500 visitas a la tienda online".
- Información: "500 visitas diarias, de las cuales el 10% finalizan en compras".
- Conocimiento: "Optimizar la página de pagos podría aumentar la conversión en un 5%".

La minería de datos actúa transformando datos en conocimiento útil.

2.3. Aplicaciones de la Minería de Datos

La minería de datos se aplica en una amplia variedad de sectores.

Principales aplicaciones:

1. Detección de fraudes (Banca y Finanzas)

Los bancos utilizan minería de datos para identificar transacciones sospechosas.

Ejemplo:

Si un cliente que normalmente hace compras en Argentina realiza una compra de alto valor en otro país, el sistema puede detectar una anomalía y bloquear la transacción.

2. Recomendaciones de productos (Retail y e-commerce)

Amazon, MercadoLibre y otras plataformas analizan el comportamiento de los usuarios para ofrecer recomendaciones personalizadas.

Ejemplo:

Si compras una laptop, el sistema te sugiere un mouse y una funda, porque otros clientes con hábitos similares hicieron compras relacionadas.

3. Minería de datos en Salud

Se usa para predecir enfermedades y personalizar tratamientos.

Ejemplo:

Un hospital puede analizar datos de miles de pacientes para predecir riesgos de enfermedades cardíacas en función de la edad, peso y antecedentes familiares.

4. Marketing y segmentación de clientes

Las empresas analizan datos de consumo para segmentar clientes en grupos homogéneos y dirigir campañas personalizadas.

Ejemplo:

Netflix analiza qué tipos de series ves y por cuánto tiempo, para recomendarte contenidos basados en tu comportamiento y en el de otros usuarios similares.

5. Minería de datos en transporte y logística

Las empresas optimizan rutas de entrega para reducir costos.

Ejemplo:

Uber usa minería de datos para predecir la demanda y ajustar sus precios en función del tráfico y la cantidad de conductores disponibles.

2.4. ¿Cómo se relaciona la minería de datos con el Machine Learning?

Muchas veces, Minería de Datos y Machine Learning son términos que se confunden.

- Minería de Datos → Descubre patrones en datos históricos.
- Machine Learning → Usa patrones descubiertos para hacer predicciones automáticas en nuevos datos.

-

Ejemplo práctico:

- Minería de datos: Un supermercado descubre que los clientes que compran leche también suelen comprar pan.
- Machine Learning: Se entrena un modelo para predecir qué productos comprará un cliente, basado en su historial de compras

.[]

Minería de Datos extrae patrones. Machine Learning usa esos patrones para predecir.

Punto clave de esta sección:

La minería de datos no solo analiza datos históricos, sino que también ayuda a predecir eventos futuros y a optimizar la toma de decisiones en distintos sectores.

Ejercicio para la Sección 2: Introducción a la Minería de Datos

Objetivo: Comprender la diferencia entre dato, información y conocimiento, y explorar cómo la minería de datos puede descubrir patrones en un conjunto de datos.

Ejercicio 1: Identificación de Datos, Información y Conocimiento

A continuación, se presentan tres situaciones. Para cada una, identifica qué es el dato, la información y el conocimiento.

1. Un banco analiza 10,000 transacciones realizadas en las últimas 24 horas. Encuentra que 2,000 transacciones provienen de cuentas recién creadas y de países donde no opera habitualmente. Basándose en esto, activa un sistema de alerta para detectar fraudes.
2. Una tienda online nota que el 70% de los clientes que compran laptops también compran fundas protectoras en un lapso de 7 días. Basado en este hallazgo, comienza a ofrecer un descuento del 10% en fundas al comprar una laptop.
3. Un hospital analiza los registros médicos de 50,000 pacientes y descubre que las personas con hipertensión tienen un 40% más de probabilidades de desarrollar enfermedades cardíacas en los próximos 10 años. Con base en esto, implementa un programa preventivo de chequeos cardíacos gratuitos para personas con hipertensión.

Instrucciones:

- Para cada caso, señala cuál es el dato, cuál la información y cuál el conocimiento.
- Explica en pocas palabras cómo la minería de datos ayuda en cada situación.

Ejemplo de respuesta (para el primer caso):

- Dato: "10,000 transacciones en 24 horas".
- Información: "2,000 transacciones provienen de cuentas nuevas y de países inusuales".
- Conocimiento: "Si una transacción proviene de una cuenta nueva y de un país inusual, es más probable que sea fraudulenta".
- Papel de la Minería de Datos: Detectó un patrón que permite mejorar la seguridad del banco.

Sección 3: Proceso KDD y CRISP-DM

3. Proceso de Descubrimiento de Conocimiento en Bases de Datos (KDD)

El Proceso KDD (Knowledge Discovery in Databases) es una metodología ampliamente utilizada para extraer conocimiento a partir de grandes volúmenes de datos.

Se compone de cinco fases fundamentales:

1. Selección de datos

- Se eligen los datos relevantes para el problema a analizar.
- Ejemplo: Un e-commerce selecciona datos de compras de los últimos 2 años para analizar patrones de compra.

2. Preprocesamiento y limpieza de datos

- Se eliminan valores erróneos, duplicados o inconsistentes.
- Ejemplo: Un hospital elimina registros con edades negativas o fechas imposibles.

3. Transformación de datos

- Se convierten los datos en un formato adecuado para el análisis.
- Ejemplo: Convertir datos de texto en valores numéricos para análisis en Machine Learning.

4. Minería de datos

- Se aplican algoritmos para encontrar patrones o tendencias.
- Ejemplo: Identificar clientes con alto riesgo de abandonar un servicio mediante análisis de comportamiento.

5. Evaluación e interpretación

- Se analizan los patrones obtenidos para validar su utilidad.
- Ejemplo: Un banco revisa si los patrones descubiertos ayudan a detectar fraudes.

Ejemplo en Python – Inspección de Datos en Pandas

```
import pandas as pd

# Cargar datos desde un archivo CSV
df = pd.read_csv("ventas.csv")

# Mostrar primeras filas

print(df.head())

# Verificar si hay valores faltantes

print(df.isnull().sum())

# Resumen estadístico de los datos numéricos

print(df.describe())
```

Este código muestra cómo seleccionar, limpiar y explorar datos, cubriendo las primeras fases del proceso KDD.

3.1. CRISP-DM: Modelo Estándar para la Minería de Datos

Además del proceso KDD, otro modelo ampliamente utilizado en la industria es CRISP-DM (Cross Industry Standard Process for Data Mining).

Fases de CRISP-DM:

1. Comprensión del negocio: Se definen objetivos y necesidades del proyecto.
2. Comprensión de los datos: Se analizan fuentes de datos y calidad de la información.

3. Preparación de los datos: Se limpian y transforman datos para análisis.
4. Modelado: Se aplican algoritmos de minería de datos (clustering, regresión, árboles de decisión, etc.).
5. Evaluación: Se valida la calidad y utilidad de los modelos generados.
6. Despliegue: Se implementan los modelos en producción para uso real.

Ejemplo de Aplicación de CRISP-DM:

Una empresa de telecomunicaciones quiere predecir qué clientes van a cancelar su servicio.

- Comprensión del negocio: Entender las razones por las que los clientes abandonan.
- Comprensión de los datos: Analizar datos de facturación, llamadas al soporte y encuestas.
- Preparación de los datos: Eliminar clientes duplicados y corregir valores inconsistentes.
- Modelado: Aplicar Machine Learning para predecir clientes propensos a cancelar.
- Evaluación: Medir precisión del modelo y ajustarlo según resultados.
- Despliegue: Implementar un sistema de alertas para retener clientes en riesgo.

Comparación entre KDD y CRISP-DM

Característica	KDD	CRISP-DM

Enfoque	Más académico	Más empresarial
Fases	5 fases	6 fases
Objetivo	Descubrimiento de patrones	Aplicación práctica en negocios
Nivel de detalle	Menos estructurado	Más detallado en cada fase

Ejercicio para la Sección 3.

Objetivo: Aplicar los conceptos del proceso KDD y CRISP-DM en situaciones reales y con datos sencillos.

Ejercicio 1: Identificación de Fases del Proceso KDD

A continuación, se presentan cuatro situaciones. Para cada una, identifica a qué fase del proceso KDD pertenece.

1. Un equipo de científicos de datos descarga una base de datos con información sobre ventas en un supermercado.
2. Se eliminan registros con datos faltantes y se corrigen inconsistencias en los formatos de fechas.
3. Se aplica un algoritmo de agrupamiento (clustering) para segmentar clientes según su comportamiento de compra.
4. Se presentan los resultados en un informe y se analizan los patrones descubiertos.

Instrucciones:

- Asocia cada situación con una fase del proceso KDD: Selección, Preprocesamiento, Transformación, Minería de Datos o Evaluación.

Ejercicio 2: Aplicación del Modelo CRISP-DM

Un banco quiere detectar fraudes en transacciones. Relaciona cada acción con la fase de CRISP-DM correspondiente:

1. Se analizan casos anteriores de fraude y se definen los objetivos del análisis.
2. Se limpian y transforman los datos de transacciones.
3. Se prueban diferentes modelos de Machine Learning para encontrar patrones de fraude.
4. Se analiza la efectividad de los modelos y se elige el más preciso.
5. Se implementa un sistema automático que detecta fraudes en tiempo real.

Instrucciones:

- Indica a qué fase de CRISP-DM pertenece cada acción: Comprensión del Negocio, Comprensión de los Datos, Preparación de Datos, Modelado, Evaluación o Despliegue.

Sección 4: Preprocesamiento y Análisis Exploratorio de Datos

El preprocesamiento de datos es una etapa clave en minería de datos, ya que los datos en bruto suelen contener errores, valores atípicos o datos faltantes.

4.1. Limpieza de Datos

Los datos pueden contener problemas como:

- Valores nulos o faltantes → Registros sin información en algunas columnas.
- Valores duplicados → Registros repetidos que generan sesgo en el análisis.
- Valores inconsistentes → Datos incorrectos o fuera de rango (ej: edades negativas).

Ejemplo en Python – Identificación de Problemas

```
import pandas as pd
```

```
# Cargar datos ficticios

datos = {

"Nombre": ["Ana", "Luis", "Carlos", None, "Marta"],
"Edad": [25, None, 40, 35, -5],
"Salario": [50000, 55000, 60000, 58000, None]
}

df = pd.DataFrame(datos)

# Ver valores faltantes

print(df.isnull().sum())

# Ver datos fuera de rango

print(df[df["Edad"] < 0])
```

Acciones recomendadas:

- Eliminar filas con demasiados valores faltantes.
- Imputar valores faltantes con la media o la mediana.
- Corregir valores atípicos detectados.

4.2. Transformación de Datos

Después de limpiar los datos, se pueden realizar transformaciones para facilitar el análisis.

Tareas comunes:

- Normalización: Escalar valores numéricos entre 0 y 1.
- Codificación de variables categóricas: Convertir texto en valores numéricos para modelos de Machine Learning.
- Creación de nuevas variables: Extraer información relevante de datos existentes.

Ejemplo en Python – Normalización y Codificación

```
from sklearn.preprocessing import MinMaxScaler, LabelEncoder

# Normalización de la columna "Salario"
scaler = MinMaxScaler()
df["Salario_Normalizado"] = scaler.fit_transform(df[["Salario"]])

# Codificación de la columna "Nombre"
encoder = LabelEncoder()
df["Nombre_Codificado"] = encoder.fit_transform(df["Nombre"].astype(str)) print(df)
```

4.3. Análisis Exploratorio de Datos (EDA)

Antes de aplicar modelos de minería de datos, es fundamental explorar los datos con visualizaciones y estadísticas descriptivas.

Herramientas clave:

- Histogramas → Para ver la distribución de una variable.
- Boxplots → Para detectar valores atípicos.
- Mapas de calor → Para analizar correlaciones entre variables.

Ejemplo en Python – Visualización de Datos

```
import seaborn as sns

import matplotlib.pyplot as plt

# Histograma de edades
```

```
sns.histplot(df["Edad"].dropna(), bins=5, kde=True)
```

```
plt.show()
```

```
# Boxplot de salarios
```

```
sns.boxplot(x=df["Salario"])
```

```
plt.show()
```

Interpretación de Resultados:

- Un histograma ayuda a identificar sesgos en la distribución.
- Un boxplot permite ver si hay valores atípicos.

Ejercicios para la Sección 4

Ejercicio 1: Detección de Problemas en Datos

Supón que tienes el siguiente conjunto de datos de estudiantes:

Nombre Edad Nota

• -----

Juan 22 8

Sofía -3 10

Lucas 25 6

Ana 20 None

Marcos 21 4

Instrucciones:

1. **Identifica los problemas en estos datos.**
2. **¿Cómo los corregirías? (eliminar, reemplazar, corregir valores negativos, etc.).**
3. **¿Qué tipo de transformación aplicarías antes de analizar los datos?**

Ejercicio 2: Creación de Gráficos en Python

Crea un DataFrame con los siguientes datos de ventas:

Producto Precio Cantidad_Vendida

• -----

A 10 50

B 20 30

C 15 40

D 25 20

Instrucciones:

1. **Escribe código en Python para crear un histograma de precios.**
2. **Crea un gráfico de dispersión de Precio vs Cantidad_Vendida.**
3. **Interpreta los resultados de los gráficos.**

Ejercicio Integrador – Clase 1: Introducción a la Minería de Datos

Objetivo: Aplicar los conceptos aprendidos sobre Minería de Datos, KDD, CRISP-DM, Preprocesamiento y Exploración de Datos en un ejercicio práctico.

Contexto del Problema

Una empresa de telecomunicaciones quiere analizar los datos de sus clientes para identificar patrones de consumo y detectar posibles cancelaciones de servicio. Se tiene un conjunto de datos con la siguiente información:

Cliente Edad Plan Contratado Consumo de Datos (GB) Reclamos en el Último Año Estado de Cuenta (Pagado/Moroso)

• -----

1 25 Básico 5 1 Pagado

2 40 Premium 50 0 Pagado

3	32	Estándar	10	2	Moroso
4	22	Básico	7	3	Moroso
5	35	Premium	45	0	Pagado

Tareas a Realizar

1. Identificación de Fases en KDD y CRISP-DM

- ¿Cómo aplicaría el proceso KDD a estos datos?
- ¿Qué pasos seguirías antes de extraer patrones?
- ¿Qué fase de CRISP-DM corresponde a cada acción en este análisis?

2. Preprocesamiento de Datos

- ¿Qué problemas podrías encontrar en los datos?
- ¿Hay valores inconsistentes o que deban transformarse?
- ¿Cómo manejarías los valores categóricos (Plan Contratado, Estado de Cuenta)?
- Escribe el código en Python para cargar los datos en un DataFrame de Pandas y verificar valores faltantes o duplicados.

3. Exploración de Datos

- Genera un histograma del consumo de datos (GB) para analizar la distribución.
- Crea un gráfico de barras del "Estado de Cuenta" para ver cuántos clientes son morosos.

4. Interpretación de Resultados

- ¿Qué tendencias observas en los datos?
- ¿Hay alguna relación entre el consumo de datos y el estado de cuenta?
- ¿Los clientes morosos tienen características en común?

Código Base para Resolver el Ejercicio

Aquí tienes un punto de partida para analizar los datos en Python:

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

# Crear DataFrame con los datos del ejercicio

datos = {
    "Cliente": [1, 2, 3, 4, 5],
    "Edad": [25, 40, 32, 22, 35],
    "Plan": ["Básico", "Premium", "Estándar", "Básico", "Premium"],
    "Consumo_GB": [5, 50, 10, 7, 45],
    "Reclamos": [1, 0, 2, 3, 0],
    "Estado_Cuenta": ["Pagado", "Pagado", "Moroso", "Moroso", "Pagado"]
}

df = pd.DataFrame(datos)

# Inspeccionar los datos

print(df.info())

print(df.describe())

# Ver distribución de consumo de datos

sns.histplot(df["Consumo_GB"], bins=5, kde=True)
```

```
plt.show()

# Gráfico de barras de Estado de Cuenta

sns.countplot(x=df["Estado_Cuenta"])

plt.show()
```

Publicación en el Foro

1. Comparte tu análisis en el foro de la clase con:

- Breve resumen de tu interpretación de los datos.
- Explicación de qué fase del KDD y CRISP-DM consideraste más relevante.
- Capturas o gráficos de los resultados obtenidos.

2. Comenta y participa en la publicación de otro compañero:

- Ofrece una observación o mejora sobre su análisis.
- Discute si hay otras interpretaciones posibles.
- Si encontraste un patrón diferente, compáralo con el de tu compañero.

Objetivo de la participación: Aprender desde distintas perspectivas y consolidar el conocimiento aplicando minería de datos en la práctica.

Bibliografía

1. Han, J., Kamber, M., & Pei, J. (2012). **Data Mining: Concepts and Techniques (3rd ed.)**. Elsevier. ISBN: 978-0-12-381479-1.
2. Hernández Orallo, J., Ramírez Quintana, M. J., & Ferri Ramírez, C. (2004). **Introducción a la Minería de Datos**. Pearson-Prentice Hall.
3. Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J. (2017). **Data Mining: Practical Machine Learning Tools and Techniques**. Morgan Kaufmann Publishers.
4. Layton, R. (2017). **Learning Data Mining with Python (2nd ed.)**. Packt Publishing.
5. Python Software Foundation. (n.d.). **Welcome to Python.org**. Recuperado el [fecha de consulta], de <https://www.python.org/>
6. The pandas development team. (n.d.). **Pandas: Powerful Python Data Analysis Toolkit**. Recuperado el [fecha de consulta], de <https://pandas.pydata.org/>
7. Project Jupyter. (n.d.). **Project Jupyter**. Recuperado el [fecha de consulta], de <https://jupyter.org/>

2. Tipos, Limpieza y Transformacion

Clase 2: Tipos de Datos, Limpieza y Transformación de Datos

1. Introducción

Objetivo de la Clase

El éxito de cualquier proyecto de minería de datos depende en gran medida de la calidad de los datos utilizados. En esta clase, nos enfocaremos en la importancia de la limpieza y preprocesamiento de datos para garantizar análisis precisos y modelos eficientes.

Al finalizar esta clase, deberás ser capaz de:

- Identificar y clasificar tipos de datos en distintos conjuntos de datos.
- Comprender la importancia de la limpieza de datos y aplicar técnicas para detectar valores faltantes, inconsistencias y valores atípicos.
- Realizar transformaciones en los datos para mejorar su interpretación y su uso en modelos de minería de datos.
- Aplicar herramientas de análisis exploratorio en Python para evaluar la calidad de un conjunto de datos.

1.1. Evolución Histórica y Desafíos del Volumen de Datos

En las últimas décadas, el crecimiento exponencial de los datos ha cambiado la forma en que las organizaciones gestionan la información.

Evolución de los datos en la historia:

1. Bases de datos tradicionales (años 70-90): Los sistemas almacenaban información en bases de datos relacionales estructuradas (SQL).
2. Big Data y la explosión digital (2000-presente): El auge de Internet, redes sociales, sensores IoT y la digitalización de documentos generaron volúmenes masivos de datos estructurados y no estructurados.

3. Minería de Datos y Machine Learning: La disponibilidad de datos llevó al desarrollo de herramientas avanzadas para extraer conocimiento mediante algoritmos y modelos predictivos.

Desafíos actuales en la gestión de datos:

- Volumen: Manejar cantidades masivas de datos (Big Data).
- Variedad: Integración de datos estructurados (tablas SQL) con datos no estructurados (imágenes, audios, redes sociales).
- Veracidad: Detección de datos incorrectos, sesgados o duplicados.
- Velocidad: Procesamiento de datos en tiempo real (ejemplo: detección de fraudes bancarios).

Ejemplo Real:

Un hospital puede tener datos de pacientes en bases SQL, imágenes médicas en DICOM, registros de voz en notas médicas y documentos en PDF. Para analizarlos, es necesario aplicar técnicas de preprocesamiento específicas.

1.2. Importancia del Preprocesamiento de Datos

Antes de aplicar algoritmos de minería de datos o Machine Learning, es fundamental preparar los datos. Un conjunto de datos sin preprocesar puede contener:

- Valores faltantes → Información incompleta.
- Valores atípicos (outliers) → Datos extremos que pueden sesgar el análisis.
- Valores inconsistentes → Incoherencias en nombres, formatos o unidades.
- Duplicados → Registros repetidos que afectan la precisión de los modelos.

Ejemplo de Problemas en Datos

ID Nombre Edad Fecha de Registro Ingresos Estado

•	-----	-----	-----	-----	-----
1	Juan	32	10/05/2023	45000.0	Activo
2	María	-28	15-06-2023	55000.0	ACTIVO
3	Pedro	40	20/02/2023	None	Inactivo
4	Juan	32	10/05/2023	45000.0	Activo

Problemas detectados:

- Edad negativa (error en la entrada de datos).
- Formato inconsistente en fechas.
- Valor faltante en ingresos.
- Estado inconsistente ("Activo" vs. "ACTIVO").
- Registro duplicado (Juan aparece dos veces).

Conclusión: Antes de cualquier análisis, debemos limpiar y transformar los datos para evitar sesgos o errores en los modelos.

1.3. Flujo de Trabajo en Minería de Datos

El proceso de minería de datos sigue un flujo estructurado para asegurar la calidad de los datos. Dos enfoques comunes son KDD (Knowledge Discovery in Databases) y CRISP-DM (Cross Industry Standard Process for Data Mining).

Fases del Proceso KDD

1. Selección de datos → ¿Qué datos son relevantes para el análisis?
2. Preprocesamiento → Limpieza de datos, eliminación de duplicados, corrección de errores.
3. Transformación → Normalización, codificación de variables categóricas, generación de nuevas variables.
4. Minería de Datos → Aplicación de modelos de clasificación, regresión o clustering.
5. Evaluación e Interpretación → Validación de los resultados obtenidos.

Modelo CRISP-DM

Un enfoque más estructurado utilizado en la industria que organiza el análisis en seis fases:

1. Comprensión del negocio → Definir el problema y los objetivos del análisis.
2. Comprensión de los datos → Exploración inicial, identificación de errores.
3. Preparación de los datos → Limpieza, transformación y selección de variables.
4. Modelado → Aplicación de algoritmos de minería de datos.
5. Evaluación → Validación del modelo y ajuste de parámetros.
6. Despliegue → Implementación del modelo en producción.

Ejemplo CRISP-DM aplicado a un Banco

- Comprensión del negocio: Reducir la tasa de morosidad en préstamos.
- Comprensión de los datos: Analizar historiales de pago, ingresos y edad de clientes.
- Preparación: Normalizar ingresos, codificar estados de cuenta (Pagado/Moroso).
- Modelado: Usar algoritmos de clasificación para predecir clientes de riesgo.
- Evaluación: Validar con datos de prueba y ajustar el modelo.
- Despliegue: Implementar en el sistema del banco para evaluar nuevos clientes.

Ejercicio para la Sección 1

Objetivo: Comprender la importancia del pre procesamiento y clasificación de datos.

Ejercicio 1: Identificación y Diagnóstico de Problemas en Datos

Dado el siguiente conjunto de datos de clientes de un banco:

Cliente Edad Salario Estado Civil Crédito Aprobado

- -----

1	25	50000	Soltero	Sí
2	-40	60000	Casado	No
3	32	70000	Divorciado	Sí
4	22	None	Soltero	No
5	35	80000	Viudo	Sí

Instrucciones:

1. Identifica los problemas en los datos.
2. Propón una estrategia de limpieza.
3. Escribe un código en Python para detectar valores faltantes, atípicos y duplicados.

Ejemplo de Código en Python

```
import pandas as pd

# Crear DataFrame

datos = {
    "Cliente": [1, 2, 3, 4, 5],
    "Edad": [25, -40, 32, 22, 35],
    "Salario": [50000, 60000, 70000, None, 80000],
    "Estado_Civil": ["Soltero", "Casado", "Divorciado", "Soltero", "Viudo"],

    "Credito_Aprobado": ["Sí", "No", "Sí", "No", "Sí"]
}
df = pd.DataFrame(datos)
# Ver problemas en los datos

print(df.isnull().sum()) # Valores faltantes

print(df[df["Edad"] < 0]) # Valores atípicos
```

2. Tipos de Datos y su Clasificación

La identificación y clasificación de los datos es un paso clave en minería de datos, ya que determina qué técnicas de limpieza y transformación pueden aplicarse.

2.1. Tipos de Datos según su Estructura

Los datos pueden organizarse de diferentes maneras según su formato y origen.

Tipo de Datos	Descripción	Ejemplo
• -----		
Estructurados	Datos organizados en filas y columnas. Se almacenan en bases de datos relacionales.	Tabla de clientes con nombres, edades y saldos bancarios.
No estructurados	Datos sin formato predefinido, difíciles de almacenar en bases SQL.	Imágenes, videos, audios, correos electrónicos.
Semiestructurados	Datos que tienen etiquetas o metadatos que facilitan su organización.	Archivos JSON, XML, documentos HTML.

Ejemplo en Python – Cargando diferentes tipos de datos

```
import pandas as pd

# Cargar datos estructurados (desde CSV)
df_csv = pd.read_csv("clientes.csv")
# Cargar datos semiestructurados (JSON)
df_json = pd.read_json("clientes.json")
# Inspeccionar los datos

print(df_csv.head())

print(df_json.head())
```

Conclusión: Dependiendo del tipo de dato, se requieren herramientas específicas para su almacenamiento y procesamiento.

2.2. Tipos de Datos según su Naturaleza

Los datos también pueden clasificarse según su naturaleza matemática y uso en modelos analíticos.

Tipo de Dato	Descripción	Ejemplo
	• -----	
Númericos	Valores que representan cantidades medibles.	Precio, peso, temperatura.
Catégoricos	Representan etiquetas o nombres de categorías.	Género (Masculino/Femenino), Estado Civil.
Ordinales	Categorías con un orden lógico, pero sin una distancia uniforme entre ellas.	Nivel educativo (Primaria, Secundaria, Universitaria).

Ejemplo en Python – Identificación de Tipos de Datos en un DataFrame

```
import pandas as pd

# Crear un DataFrame con distintos tipos de datos
df = pd.DataFrame({
    "Edad": [25, 30, 35, 40, 45],
    "Ingreso": [50000, 60000, 55000, 75000, 90000],
    "Estado_Civil": ["Soltero", "Casado", "Viudo", "Casado", "Soltero"],

    "Nivel_Educativo": ["Secundaria", "Universitaria", "Secundaria", "Primaria", "Universitaria"]
})
# Inspeccionar los tipos de datos

print(df.dtypes)
```

Salida esperada:

```
Edad int64
Ingreso int64
Estado_Civil object
Nivel_Educativo object
dtype: object
```

Conclusión:

- Los datos numéricos pueden usarse directamente en modelos de predicción.
- Los datos catégoricos deben transformarse antes de usarse en algoritmos de Machine Learning.

2.3. Transformación de Datos Catégoricos

Algunos algoritmos no pueden trabajar con datos catégoricos, por lo que es necesario convertirlos en datos numéricos.

2.3.1. Codificación Ordinal

Usada cuando las categorías tienen un orden específico.

Ejemplo – Convertir Nivel Educativo en Números

```
from sklearn.preprocessing import OrdinalEncoder

# Definir orden de categorías

orden_educativo = ["Primaria", "Secundaria", "Universitaria"]

# Aplicar codificación ordinal
encoder = OrdinalEncoder(categories=orden_educativo)
df["Nivel_Educativo_Cod"] = encoder.fit_transform(df[["Nivel_Educativo"]])
print(df[["Nivel_Educativo", "Nivel_Educativo_Cod"]])
```

Salida esperada:

```
Nivel_Educativo Nivel_Educativo_Cod
0 Secundaria 1.0
1 Universitaria 2.0
2 Secundaria 1.0
3 Primaria 0.0
4 Universitaria 2.0
```

Útil para datos como: tamaño de ropa (S, M, L), satisfacción del cliente (Bajo, Medio, Alto).

2.3.2. Codificación One-Hot Encoding

Se usa cuando las categorías no tienen un orden jerárquico.

Ejemplo – Convertir Estado Civil en Variables Binarias

```
from sklearn.preprocessing import OneHotEncoder

# Aplicar One-Hot Encoding
encoder = OneHotEncoder(sparse=False, drop="first")
encoded_data = encoder.fit_transform(df[["Estado_Civil"]])
# Convertir a DataFrame
df_encoded = pd.DataFrame(encoded_data, columns=encoder.get_feature_names_out(["Estado_Civil"]))
df = pd.concat([df, df_encoded], axis=1)
print(df.head())
```

Salida esperada:

```
Estado_Civil Estado_Civil_Casado Estado_Civil_Viudo
0 Soltero 0 0
1 Casado 1 0
2 Viudo 0 1
3 Casado 1 0
4 Soltero 0 0
```

Útil para datos como: País, Tipo de Producto, Categoría de Cliente.[]

Ejercicio para la Sección 2

Objetivo: Aplicar la clasificación y transformación de datos categóricos.

Ejercicio 1: Clasificación y Transformación de Datos

Supongamos que tenemos el siguiente conjunto de datos:

ID	Nombre	Edad	Categoría Cliente	Estado Civil
1	Juan	32	Oro	Soltero
2	María	28	Plata	Casado
3	Pedro	40	Platino	Viudo
4	Ana	35	Oro	Casado

Instrucciones:

1. Clasifica cada columna según su tipo (numérico, categórico, ordinal).
2. Indica qué método de transformación aplicarías para convertir "Categoría Cliente" y "Estado Civil" en datos numéricos.

3. Escribe un código en Python para cargar estos datos en un DataFrame y aplicar la transformación adecuada.

Pista para la transformación en Python

```
import pandas as pd

from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder

# Crear DataFrame

data = {
    "ID": [1, 2, 3, 4],
    "Nombre": ["Juan", "María", "Pedro", "Ana"],
    "Edad": [32, 28, 40, 35],
    "Categoria_Cliente": ["Oro", "Plata", "Platino", "Oro"],

    "Estado_Civil": ["Soltero", "Casado", "Viudo", "Casado"]
}
df = pd.DataFrame(data)
# Codificación ordinal para "Categoría Cliente"

orden_categoria = ["Plata", "Oro", "Platino"]
encoder_ordinal = OrdinalEncoder(categories=orden_categoria)
df["Categoria_Cod"] = encoder_ordinal.fit_transform(df[["Categoria_Cliente"]])
# One-Hot Encoding para "Estado Civil"
encoder_onehot = OneHotEncoder(sparse=False, drop="first")
encoded_estado = encoder_onehot.fit_transform(df[["Estado_Civil"]])
df_encoded = pd.DataFrame(encoded_estado, columns=encoder_onehot.get_feature_names_out(["Estado_Civil"]))
df = pd.concat([df, df_encoded], axis=1)
print(df.head())
```

Sección 4: Introducción a la Transformación y Análisis Exploratorio de Datos

Objetivo: Entender qué es la transformación de datos y por qué es fundamental en minería de datos, además de realizar un primer acercamiento al análisis exploratorio.

4.1. ¿Qué es la Transformación de Datos?

Después de limpiar los datos, en muchos casos es necesario modificarlos para mejorar su calidad y usabilidad. Esto se conoce como transformación de datos y es un paso clave en minería de datos y aprendizaje automático.

Ejemplos comunes de transformación:

Conversión de tipos de datos (de texto a numérico, de fecha a componentes de tiempo).

Normalización y escalado (para ajustar valores numéricos a una escala común).

Codificación de variables categóricas (convertir texto en valores numéricos).

Creación de nuevas variables a partir de variables existentes (ejemplo: calcular el IMC a partir de peso y altura).

Ejemplo en Python – Conversión de Datos de Texto a Números

```
import pandas as pd

from sklearn.preprocessing import LabelEncoder

# Datos originales
df = pd.DataFrame({
    "Cliente": ["A", "B", "C", "D"],

    "Estado_Cuenta": ["Pagado", "Moroso", "Pagado", "Moroso"]
})
# Convertir texto a números
encoder = LabelEncoder()
df["Estado_Cuenta_Num"] = encoder.fit_transform(df["Estado_Cuenta"])
print(df)
```

Salida esperada:

```
Cliente Estado_Cuenta Estado_Cuenta_Num
0 A Pagado 1
1 B Moroso 0
2 C Pagado 1
3 D Moroso 0
```

Conclusión: Transformar variables categóricas es clave para usarlas en modelos predictivos.

4.2. ¿Qué es el Análisis Exploratorio de Datos (EDA)?

El Análisis Exploratorio de Datos (Exploratory Data Analysis, EDA) es el proceso de investigar un conjunto de datos mediante estadísticas descriptivas y visualizaciones antes de aplicar modelos de minería de datos.

Objetivos del EDA:

- Identificar patrones generales en los datos.
- Detectar valores atípicos o problemas en los datos.
- Visualizar distribuciones y correlaciones.

Ejemplo en Python – Análisis Rápido con Pandas

```
import pandas as pd

# Cargar datos
df = pd.DataFrame({
    "Edad": [25, 32, 40, 35, 50],
    "Ingresos": [30000, 45000, 60000, 55000, 80000]
})

# Resumen estadístico

print(df.describe())

# Correlación entre variables

print(df.corr())
```

Salida esperada: Estadísticas descriptivas como media, mediana, percentiles y correlaciones entre variables.

Conclusión: El EDA ayuda a entender la estructura de los datos antes de aplicar modelos.

Ejercicio para la Sección 4

Objetivo: Aplicar conceptos básicos de transformación y análisis exploratorio.

Ejercicio 1: Exploración y Transformación de Datos

Dado el siguiente conjunto de datos:

ID Nombre Edad Categoría Cliente Ingresos

• -----

1	Juan	32	Oro		45000
2	María	28	Plata		55000
3	Pedro	40	Platino		70000
4	Ana	35	Oro		60000

Instrucciones:

1. Calcula estadísticas descriptivas (media, mediana, valores máximos y mínimos).
2. Convierte la columna "Categoría Cliente" en una variable numérica.
3. Calcula la correlación entre Edad e Ingresos.

Pista para la solución en Python

```
import pandas as pd

from sklearn.preprocessing import OrdinalEncoder

# Crear DataFrame
df = pd.DataFrame({
    "ID": [1, 2, 3, 4],
    "Nombre": ["Juan", "María", "Pedro", "Ana"],
    "Edad": [32, 28, 40, 35],
    "Categoria_Cliente": ["Oro", "Plata", "Platino", "Oro"],
    "Ingresos": [45000, 55000, 70000, 60000]
})

# Estadísticas descriptivas

print(df.describe())

# Codificación ordinal de "Categoría Cliente"

orden_categoria = ["Plata", "Oro", "Platino"]
encoder = OrdinalEncoder(categories=orden_categoria)
df["Categoria_Cod"] = encoder.fit_transform(df[["Categoria_Cliente"]])
# Calcular correlación

print(df[["Edad", "Ingresos"]].corr())

print(df)
```

Instancia de evaluación — Actividad Obligatoria (Clase 2)

Título: Limpieza, Transformación y Análisis Exploratorio de Datos con LLMs **Objetivo:** Aplicar identificación de tipos de datos, limpieza, transformación y análisis exploratorio, integrando el uso de un LLM (p. ej., ChatGPT o Gemini) como apoyo metodológico.

Entrega en Moodle

- Subir dos archivos:
- AO1_Clase2_Apellido_Nombre.ipynb (el notebook debe ejecutar inicio a fin sin errores; "Restart & Run All").
- AO1_Clase2_Apellido_Nombre.pdf (exportación del mismo notebook, con todas las salidas visibles).
- Incluir en el notebook:
- Clasificación de tipos de datos y criterios de manejo.
- Limpieza (faltantes, consistencias en categóricas, outliers en Salario).
- Transformación (codificación One-Hot u ordinal; normalización/estandarización de Salario; "Años hasta jubilación" = 65 – edad).
- EDA con al menos dos visualizaciones y breve interpretación.
- Uso de LLM: 1–2 prompts y síntesis (3–5 líneas por prompt) de lo aportado, con validación propia.
- La fecha y hora límite se rigen por lo indicado en Moodle.

Criterio de Evaluación.

Mediante la siguiente rúbrica en la plataforma moodle:

Criterio	No logrado. 0 pts	Parcialmente logrado. 1 pto	Completamente logrado. 2 puntos.
1. Tipos de datos y criterio de manejo	Clasificación incorrecta o incompleta; sin justificación del manejo.	Clasificación mayormente correcta con omisiones; justificación general.	Clasificación completa y correcta; justifica el manejo y su impacto en el flujo de trabajo.
2. Limpieza (faltantes, inconsistencias, outliers en Salario)	Faltantes sin tratamiento; categóricas inconsistentes; outliers no detectados o mal tratados.	Trata faltantes y categóricas con resultados parciales; detecta outliers con justificación limitada.	Trata faltantes con criterio; unifica categóricas; detecta y trata outliers con método justificado (IQR/z-score/percentiles) e informa umbrales.

Criterio	No logrado. 0 pts	Parcialmente logrado. 1 pto	Completamente logrado. 2 puntos.
3. Transformación (encoding, normalización, variable nueva)	Codificación inadecuada o ausente; normalización ausente; "Años hasta jubilación" incorrecta o ausente.	Codificación y normalización con errores menores o sin justificación; variable creada con detalles a corregir.	Codificación apropiada (One-Hot u ordinal) y normalización/estandarización justificadas; "Años hasta jubilación" correcta; decisiones explicadas.
4. EDA (gráficas e interpretación)	Menos de 2 visualizaciones o irrelevantes; sin interpretación.	≥ 2 visualizaciones pertinentes; interpretación básica.	≥ 2 visualizaciones pertinentes con interpretación clara de tendencias, dispersiones y relaciones; hallazgos fundamentados.
5. Claridad, reproducibilidad y uso de LLM	Notebook no ejecuta de inicio a fin; sin evidencia útil de LLM; documentación deficiente.	Notebook ejecuta con fricciones menores; prompts incluidos con integración limitada; documentación suficiente.	Notebook ejecuta completamente; decisiones documentadas; prompts y síntesis del LLM integrados con validación propia; nomenclatura de archivos correcta.

Situación problemática.

Contexto del Problema

Una empresa de recursos humanos quiere analizar un conjunto de datos de empleados para identificar patrones en sueldos, edades y categorías laborales.

El dataset contiene algunos problemas comunes, como valores faltantes, datos inconsistentes y variables categóricas sin codificar. Además, la empresa quiere entender la relación entre edad, salario y nivel educativo.

Tareas a Realizar

1. Identificación de Tipos de Datos

Clasifica cada columna según su tipo (numérico, categórico, ordinal, fecha) y explica cómo se debería manejar cada una.

2. Limpieza de Datos

- Detecta y corrige valores faltantes en las columnas necesarias.
- Identifica y elimina valores atípicos en la columna "Salario".
- Corrige inconsistencias en los datos categóricos (ejemplo: "Licenciado" y "licenciado" deben ser tratados como el mismo valor).

3. Transformación de Datos

- Convierte las variables categóricas en numéricas utilizando codificación ordinal o One-Hot Encoding.
- Aplica normalización al salario para facilitar comparaciones.
- Crea una nueva variable "Años hasta jubilación" suponiendo que la edad de retiro es 65 años.

4. Análisis Exploratorio con LLM

Usa ChatGPT o una herramienta LLM para ayudarte en alguna de estas tareas:

- Generar un resumen de los datos.
- Explicar cómo manejar valores faltantes.
- Sugerir formas de visualizar los datos.

Ejemplo de consulta a ChatGPT:

"Tengo un dataset con sueldos y edades, ¿cómo puedo detectar y eliminar outliers en Python?"

Comparte en el foro la consulta que hiciste y la respuesta que te dio el LLM.

Dataset para el Ejercicio

ID Nombre Edad Nivel Educativo Salario Categoría Estado

• -----

1	Juan	32	Licenciado	50000.0	Junior	Activo
2	María	-28	Ingeniero	60000.0	Senior	ACTIVO
3	Pedro	40	Doctorado	None	Senior	Inactivo
4	Ana	35	Licenciado	55000.0	Junior	Activo
5	Luis	50	Doctorado	200000.0	Manager	Inactivo

Pistas para Resolverlo en Python

Código base para comenzar el análisis:

```
import pandas as pd

from sklearn.preprocessing import MinMaxScaler, OrdinalEncoder

# Cargar los datos
df = pd.DataFrame({
    "ID": [1, 2, 3, 4, 5],
    "Nombre": ["Juan", "María", "Pedro", "Ana", "Luis"],
    "Edad": [32, -28, 40, 35, 50],
    "Nivel_Educativo": ["Licenciado", "Ingeniero", "Doctorado", "Licenciado", "Doctorado"],
    "Salario": [50000, 60000, None, 55000, 200000],
    "Categoría": ["Junior", "Senior", "Senior", "Junior", "Manager"],

    "Estado": ["Activo", "ACTIVO", "Inactivo", "Activo", "Inactivo"]
})

# 1 Corrección de edad negativa
df["Edad"] = df["Edad"].abs()
# 2 Imputación del salario faltante con la mediana
df["Salario"].fillna(df["Salario"].median(), inplace=True)
# 3 Detectar y eliminar valores atípicos con IQR
Q1, Q3 = df["Salario"].quantile([0.25, 0.75])
IQR = Q3 - Q1
df = df[~((df["Salario"] < (Q1 - 1.5 * IQR)) | (df["Salario"] > (Q3 + 1.5 * IQR)))]
# 4 Codificación ordinal para "Nivel Educativo"
encoder_ordinal = OrdinalEncoder(categories=[["Licenciado", "Ingeniero", "Doctorado"]])
df["Nivel_Cod"] = encoder_ordinal.fit_transform(df[["Nivel_Educativo"]])
# 5 Normalización del salario
scaler = MinMaxScaler()
df["Salario_Normalizado"] = scaler.fit_transform(df[["Salario"]])
# 6 Creación de nueva variable "Años hasta jubilación"
df["Años_Hasta_Jubilación"] = 65 - df["Edad"]
print(df)
```

Bibliografía

1. Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques* (3rd ed.). Elsevier. ISBN: 978-0-12-381479-1.
2. Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J. (2017). *Data Mining: Practical Machine Learning Tools and Techniques*. Morgan Kaufmann Publishers.
3. Hernández Orallo, J., Ramírez Quintana, M. J., & Ferri Ramírez, C. (2004). *Introducción a la Minería de Datos*. Pearson-Prentice Hall.
4. Provost, F., & Fawcett, T. (2013). *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
5. Python Software Foundation. (n.d.). Welcome to Python.org. Recuperado el [fecha de consulta], de <https://www.python.org/>
6. The pandas development team. (n.d.). Pandas: Powerful Python Data Analysis Toolkit. Recuperado el [fecha de consulta], de <https://pandas.pydata.org/>
7. Seaborn Development Team. (n.d.). Seaborn: Statistical Data Visualization. Recuperado el [fecha de consulta], de <https://seaborn.pydata.org/>
8. Project Jupyter. (n.d.). Project Jupyter. Recuperado el [fecha de consulta], de <https://jupyter.org/>
9. Bengfort, B., Bilbro, R., & Ojeda, T. (2018). *Applied Text Analysis with Python: Enabling Language-Aware Data Products with Machine Learning*. O'Reilly Media.

3. EDA, Analisis Multivariado

Clase 3: Análisis Exploratorio y Análisis Multivariado de Datos

1. PRESENTACIÓN

Objetivo de la Clase

El análisis exploratorio de datos (EDA) es un paso fundamental en la minería de datos y en cualquier proyecto de ciencia de datos. Nos ayuda a comprender la distribución, relaciones y patrones de los datos antes de aplicar modelos predictivos o de clustering.

En esta clase, aprenderemos a:

- Identificar tipos de análisis exploratorio y cuándo utilizarlos.
- Aplicar técnicas de análisis univariado y multivariado.
- Usar Python y bibliotecas como Pandas, Matplotlib y Seaborn para realizar exploraciones visuales y estadísticas.
- Integrar herramientas LLMs (Modelos de Lenguaje de Gran Escala) para generar insights sobre los datos.

Importancia del Análisis Exploratorio

- Permite detectar valores atípicos, errores o datos faltantes.
- Ayuda a identificar relaciones entre variables antes de modelar.
- Facilita la selección de métodos de transformación de datos.
- Proporciona insights valiosos sin necesidad de modelos complejos.

Ejemplo real: Antes de predecir qué clientes de un banco dejarán de pagar un crédito, primero debemos analizar edad, ingresos, historial de pagos y comportamiento de compra para encontrar correlaciones y patrones.

2. Tipos de Análisis Exploratorio de Datos

Existen dos tipos principales de análisis exploratorio:

Tipo de Análisis	Descripción	Ejemplo
Análisis Univariado	Examina una variable a la vez para entender su distribución.	Histograma de edades de clientes.
Análisis Multivariado	Analiza relaciones entre múltiples variables.	Correlación entre edad e ingresos en un dataset de clientes.

- **Análisis Univariado:** Se usa para entender la distribución de una variable y detectar valores atípicos o sesgos.
- **Análisis Multivariado:** Se usa para identificar relaciones y patrones entre dos o más variables.

Ejemplo en Python – Exploración Univariada y Multivariada

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Cargar datos de ejemplo
df = pd.DataFrame({
    "Edad": [25, 32, 40, 35, 50, 45, 29, 37, 41, 60],
    "Ingresos": [30000, 45000, 60000, 55000, 80000, 75000, 40000, 58000, 72000, 90000]
})

# Análisis Univariado: Histograma de edades
sns.histplot(df["Edad"], bins=5, kde=True)
plt.title("Distribución de Edades")
plt.show()

# Análisis Multivariado: Diagrama de dispersión entre Edad e Ingresos
sns.scatterplot(x=df["Edad"], y=df["Ingresos"])
plt.title("Relación entre Edad e Ingresos")
plt.show()
```

Interpretación:

- Un histograma nos permite ver si la variable edad sigue una distribución normal o sesgada.
- Un gráfico de dispersión nos ayuda a analizar si existe correlación entre edad e ingresos.

Ejercicio para la Sección 1

Objetivo: Aplicar análisis exploratorio en un conjunto de datos real.

Ejercicio 1: Exploración de Datos de Clientes

Supongamos que tenemos un dataset con información de clientes bancarios:

Cliente	Edad	Salario	Estado de Cuenta
1	25	50000	Pagado
2	40	60000	Moroso
3	35	70000	Pagado
4	50	80000	Moroso
5	30	45000	Pagado

Instrucciones:

1. Crea un DataFrame en Python con estos datos.
2. Genera un histograma de la variable Edad.
3. Crea un gráfico de dispersión entre Edad y Salario.
4. Explica qué observas en los gráficos.

Pista para la solución en Python

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Crear DataFrame
df = pd.DataFrame({
    "Cliente":      [1, 2, 3, 4, 5],
    "Edad":         [25, 40, 35, 50, 30],
    "Salario":      [50000, 60000, 70000, 80000, 45000],
    "Estado_Cuenta": ["Pagado", "Moroso", "Pagado", "Moroso", "Pagado"]
})

# Histograma de edades
sns.histplot(df["Edad"], bins=5, kde=True)
plt.title("Distribución de Edades")
plt.show()

# Gráfico de dispersión entre Edad y Salario
sns.scatterplot(x=df["Edad"], y=df["Salario"])
plt.title("Relación entre Edad y Salario")
plt.show()
```

Reflexión final:

- ¿La variable Edad tiene una distribución normal o sesgada?
- ¿Se observa alguna correlación entre Edad y Salario?

Sección 2: Análisis Univariado y Multivariado

Objetivo: Comprender con mayor profundidad cómo analizar variables individuales y múltiples en un conjunto de datos, identificando patrones, distribuciones y relaciones.

2.1. Análisis Univariado

El análisis univariado examina una sola variable a la vez, permitiendo entender su distribución, tendencia y presencia de valores atípicos.

Técnicas Comunes

Técnica	Descripción	Ejemplo
Medidas de tendencia central	Media, mediana y moda para analizar la distribución de la variable.	¿Cuál es el salario promedio de los empleados?

Técnica	Descripción	Ejemplo
Medidas de dispersión	Rango, varianza y desviación estándar para analizar la variabilidad de los datos.	¿Cuánto varían las edades de los clientes?
Histogramas y Boxplots	Visualización de distribuciones y detección de outliers.	¿Los ingresos tienen una distribución normal?

Ejemplo en Python – Análisis Univariado

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Crear DataFrame con datos de ejemplo
df = pd.DataFrame({
    "Edad": [25, 32, 40, 35, 50, 45, 29, 37, 41, 60],
    "Ingresos": [30000, 45000, 60000, 55000, 80000, 75000, 40000, 58000, 72000, 90000]
})

# Medidas estadísticas básicas
print(df["Edad"].describe())

# Histograma para analizar la distribución
sns.histplot(df["Edad"], bins=5, kde=True)
plt.title("Distribución de Edades")
plt.show()

# Boxplot para detectar outliers
sns.boxplot(x=df["Edad"])
plt.title("Boxplot de Edades")
plt.show()
```

Conclusión:

- Los histogramas permiten analizar la forma de la distribución de los datos.
- Los boxplots ayudan a identificar valores atípicos y rangos de variabilidad.

2.2. Análisis Multivariado

El análisis multivariado estudia cómo dos o más variables se relacionan entre sí, permitiendo detectar correlaciones, patrones y estructuras ocultas.

Técnicas Comunes

Técnica	Descripción	Ejemplo
Correlación de Pearson	Evalúa la relación lineal entre dos variables numéricas.	¿La edad está relacionada con el salario?
Diagramas de dispersión	Visualización gráfica de la relación entre dos variables.	¿A mayor experiencia, mayor sueldo?
Análisis de Componentes Principales (PCA)	Reduce la dimensionalidad del dataset identificando variables más influyentes.	¿Cuáles son las características clave de los clientes?
Clustering (agrupamiento)	Segmenta datos en grupos según similitudes.	¿Podemos agrupar clientes según su perfil de compras?

Ejemplo en Python – Correlación y Diagrama de Dispersión

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Crear DataFrame
df = pd.DataFrame({
    "Edad":      [25, 32, 40, 35, 50, 45, 29, 37, 41, 60],
    "Ingresos": [30000, 45000, 60000, 55000, 80000, 75000, 40000, 58000, 72000, 90000]
})

# Matriz de correlación
print(df.corr())

# Mapa de calor de correlación
sns.heatmap(df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Mapa de Correlación")
plt.show()

# Gráfico de dispersión
sns.scatterplot(x=df["Edad"], y=df["Ingresos"])
plt.title("Relación entre Edad e Ingresos")
plt.show()
```

Conclusión:

- Si la correlación es cercana a 1, las variables tienen una relación fuerte.
- Si la correlación es cercana a 0, no hay una relación clara.
- Un diagrama de dispersión nos permite visualizar tendencias y agrupaciones.

Ejercicio para la Sección 2

Objetivo: Aplicar técnicas de análisis univariado y multivariado en un dataset real.

Ejercicio 1: Exploración de Datos Financieros

Supongamos que tenemos un conjunto de datos de empleados con la siguiente información:

Empleado	Edad	Salario	Años de Experiencia
1	25	50000	2
2	40	60000	10
3	35	70000	7
4	50	80000	15
5	30	45000	3

Instrucciones:

1. Crea un DataFrame en Python con estos datos.
2. Realiza un análisis univariado sobre la variable "Salario" (media, mediana, histograma y boxplot).
3. Calcula la correlación entre Edad y Salario.
4. Visualiza la relación con un diagrama de dispersión.

Pista para la solución en Python

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Crear DataFrame
df = pd.DataFrame({
    "Empleado": [1, 2, 3, 4, 5],
    "Edad": [25, 40, 35, 50, 30],
    "Salario": [50000, 60000, 70000, 80000, 45000],
    "Experiencia": [2, 10, 7, 15, 3]
})

# Análisis Univariado: Estadísticas básicas
print(df["Salario"].describe())

# Histograma de salarios
sns.histplot(df["Salario"], bins=5, kde=True)
plt.title("Distribución de Salarios")
plt.show()

# Boxplot para detectar outliers en Salario
sns.boxplot(x=df["Salario"])
plt.title("Boxplot de Salarios")
plt.show()

# Análisis Multivariado: Correlación entre Edad y Salario
print(df[["Edad", "Salario"]].corr())

# Gráfico de dispersión
sns.scatterplot(x=df["Edad"], y=df["Salario"])
plt.title("Relación entre Edad y Salario")
plt.show()
```

Preguntas de Reflexión

1. ¿El salario tiene una distribución normal o está sesgado?
2. ¿Existen valores atípicos en la variable Salario?
3. ¿Qué tipo de correlación hay entre Edad y Salario?

Sección 3: Introducción a PCA y Análisis de Clustering

Objetivo: Entender cómo reducir la dimensionalidad de los datos mediante Análisis de Componentes Principales (PCA) y cómo agrupar datos similares mediante Clustering (agrupamiento).

3.1. ¿Qué es el Análisis de Componentes Principales (PCA)?

El Análisis de Componentes Principales (PCA) es una técnica de reducción de dimensionalidad que transforma un conjunto de variables correlacionadas en un conjunto más pequeño de variables no correlacionadas llamadas **componentes principales**.

¿Por qué usar PCA?

- Reduce la cantidad de variables sin perder mucha información.
- Facilita la visualización de datos en múltiples dimensiones.
- Mejora la eficiencia de los modelos de Machine Learning.

Ejemplo en Python – Aplicación de PCA

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Crear DataFrame con datos ficticios
df = pd.DataFrame({
    "Edad":      [25, 32, 40, 35, 50, 45, 29, 37, 41, 60],
    "Ingresos":  [30000, 45000, 60000, 55000, 80000, 75000, 40000, 58000, 72000, 90000],
    "Experiencia": [2, 10, 7, 15, 3, 12, 4, 9, 11, 20]
})

# Normalizar los datos
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df)

# Aplicar PCA para reducir a 2 componentes
pca = PCA(n_components=2)
df_pca = pca.fit_transform(df_scaled)
df_pca = pd.DataFrame(df_pca, columns=["PC1", "PC2"])

# Visualizar los resultados
sns.scatterplot(x=df_pca["PC1"], y=df_pca["PC2"])
plt.title("Proyección de Datos en Componentes Principales")
plt.show()
```

Conclusión:

- PCA nos ayuda a visualizar datos multidimensionales en 2D o 3D.
- Las variables originales se combinan en nuevas "componentes principales" que capturan la mayor variabilidad de los datos.

3.2. ¿Qué es el Clustering y para qué se usa?

El Clustering (agrupamiento) es una técnica de aprendizaje no supervisado que busca dividir los datos en grupos homogéneos (**clusters**) según su similitud.

Ejemplo de uso en la vida real:

- Segmentación de clientes en un banco según ingresos y gastos.
- Clasificación de imágenes en grupos similares.
- Agrupamiento de documentos en categorías.

Algoritmos de Clustering Comunes

Algoritmo	Descripción	Ejemplo de Uso
K-Means	Divide los datos en k grupos mediante centroides.	Segmentación de clientes.
Hierarchical Clustering	Crea una estructura jerárquica de clusters.	Agrupamiento de genes en biología.
DBSCAN	Detecta grupos basados en densidad.	Identificación de patrones en datos espaciales.

Ejemplo en Python – K-Means Clustering

```
from sklearn.cluster import KMeans

# Aplicar K-Means con 3 clusters
kmeans = KMeans(n_clusters=3, random_state=42)
df["Cluster"] = kmeans.fit_predict(df_scaled)

# Visualizar clusters
sns.scatterplot(x=df["Ingresos"], y=df["Experiencia"], hue=df["Cluster"], palette="viridis")
plt.title("Clusters según Ingresos y Experiencia")
plt.show()
```

Conclusión:

- Cada color representa un grupo de clientes con características similares.
- El clustering permite segmentar los datos en categorías útiles para análisis posteriores.

Ejercicio para la Sección 3

Objetivo: Aplicar PCA y Clustering en un conjunto de datos real.

Ejercicio 1: Reducción de Dimensionalidad y Agrupamiento

Dado el siguiente conjunto de datos de clientes:

Cliente	Edad	Ingresos	Experiencia
1	25	50000	2
2	40	60000	10
3	35	70000	7
4	50	80000	15
5	30	45000	3

Instrucciones:

1. Normaliza los datos antes de aplicar PCA.
2. Aplica PCA para reducir a 2 componentes principales y visualiza los resultados.
3. Aplica K-Means Clustering con 2 clusters y grafica los resultados.

Pista para la solución en Python

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# Crear DataFrame
df = pd.DataFrame({
    "Cliente":    [1, 2, 3, 4, 5],
    "Edad":       [25, 40, 35, 50, 30],
    "Ingresos":   [50000, 60000, 70000, 80000, 45000],
    "Experiencia": [2, 10, 7, 15, 3]
})

# Normalización
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df[["Edad", "Ingresos", "Experiencia"]])

# Aplicar PCA
pca = PCA(n_components=2)
df_pca = pca.fit_transform(df_scaled)
df_pca = pd.DataFrame(df_pca, columns=["PC1", "PC2"])

# Aplicar K-Means Clustering
kmeans = KMeans(n_clusters=2, random_state=42)
df_pca["Cluster"] = kmeans.fit_predict(df_pca)

# Visualizar clusters
sns.scatterplot(x=df_pca["PC1"], y=df_pca["PC2"], hue=df_pca["Cluster"], palette="viridis")
plt.title("Clusters después de PCA")
plt.show()
```

Preguntas de Reflexión

1. ¿Cuánta variabilidad explican las dos primeras componentes principales?
2. ¿Cómo cambian los resultados de clustering después de aplicar PCA?
3. ¿Qué conclusiones puedes extraer de los clusters obtenidos?

Ejercicio Integrador y Publicación en el Foro

Objetivo: Aplicar todo lo aprendido en esta clase mediante un ejercicio práctico que integre análisis exploratorio, PCA y Clustering y fomente la discusión con herramientas de IA en el foro de la clase.

4.1. Ejercicio Integrador

Supongamos que estamos analizando un dataset de clientes de un banco que busca segmentar su base de clientes según sus ingresos y años de experiencia.

Dataset para el ejercicio

Cliente	Edad	Ingresos	Años de Experiencia	Estado de Cuenta
1	25	50000	2	Pagado
2	40	60000	10	Moroso
3	35	70000	7	Pagado
4	50	80000	15	Moroso
5	30	45000	3	Pagado
6	45	85000	20	Moroso
7	38	72000	12	Pagado
8	29	48000	4	Pagado

Tareas a realizar

1. **Análisis Exploratorio de Datos (EDA)**
 - Generar histogramas de Edad e Ingresos.
 - Calcular la correlación entre Edad, Ingresos y Experiencia.
 - Visualizar relaciones con diagramas de dispersión.
2. **Aplicación de PCA para Reducción de Dimensionalidad**
 - Normalizar los datos antes de aplicar PCA.
 - Calcular cuántas componentes principales explican el 90% de la varianza.
 - Visualizar los datos en 2D usando las dos primeras componentes principales.
3. **Segmentación de Clientes con Clustering**
 - Aplicar K-Means Clustering con 3 clusters sobre los datos originales y luego sobre los datos transformados por PCA.
 - Comparar los resultados y analizar si el agrupamiento cambia después de la reducción de dimensionalidad.
 - Visualizar los clusters con gráficos de dispersión en 2D.
4. **Interacción con una Herramienta de IA (LLM)**
 - Hacer una consulta a ChatGPT, Gemini u otra herramienta LLM para obtener insights sobre los clusters.
 - Ejemplo de consulta: *"Tengo datos de clientes con variables de edad, ingresos y experiencia. ¿Qué técnicas avanzadas puedo usar para mejorar la segmentación?"*
 - Comparar la respuesta con lo aprendido en la clase y reflexionar sobre su utilidad.

Código Base para la Solución

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# Crear DataFrame
df = pd.DataFrame({
    "Cliente": [1, 2, 3, 4, 5, 6, 7, 8],
    "Edad": [25, 40, 35, 50, 30, 45, 38, 29],
    "Ingresos": [50000, 60000, 70000, 80000, 45000, 85000, 72000, 48000],
    "Experiencia": [2, 10, 7, 15, 3, 20, 12, 4]
})

# 1) Análisis Exploratorio
sns.histplot(df["Ingresos"], bins=5, kde=True)
plt.title("Distribución de Ingresos")
plt.show()

# Matriz de correlación
sns.heatmap(df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Matriz de Correlación")
plt.show()

# 2) Aplicar PCA
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df[["Edad", "Ingresos", "Experiencia"]])
pca = PCA(n_components=2)
df_pca = pca.fit_transform(df_scaled)
df_pca = pd.DataFrame(df_pca, columns=["PC1", "PC2"])

# 3) Aplicar Clustering K-Means
kmeans = KMeans(n_clusters=3, random_state=42)
df_pca["Cluster"] = kmeans.fit_predict(df_pca)

# Visualizar Clustering
sns.scatterplot(x=df_pca["PC1"], y=df_pca["PC2"], hue=df_pca["Cluster"], palette="viridis")
plt.title("Clusters de Clientes después de PCA")
plt.show()
```

Publicación en el Foro

Instrucciones para compartir resultados

Publica un mensaje en el foro de la clase con los siguientes puntos:

- Un breve resumen de tu análisis exploratorio.
- Capturas o gráficos generados con los clusters de clientes.
- ¿Cómo cambió la segmentación después de aplicar PCA?
- ¿Qué insights obtuviste usando una herramienta de IA (LLM)?

Ejemplo de Publicación en el Foro

Mi Análisis sobre Segmentación de Clientes

Resumen de EDA: "Se observó que la edad y los ingresos están moderadamente correlacionados ($r=0.62$). Los clientes con más experiencia suelen tener ingresos más altos."

Resultados de Clustering: "Usando K-Means con 3 clusters, se identificaron tres segmentos de clientes: jóvenes con ingresos bajos, adultos con ingresos medios y profesionales con ingresos altos. Aplicando PCA, se redujo la dimensionalidad y los clusters se visualizaron mejor en un espacio 2D."

Consulta a ChatGPT: "Consulté a ChatGPT sobre métodos avanzados de segmentación y sugirió probar DBSCAN para detectar posibles outliers. También recomendó analizar la varianza de cada cluster para validar su coherencia."

Participación en la Publicación de un Compañero

Comenta en la publicación de otro estudiante:

- **Pregunta:** ¿Los resultados hubieran cambiado si usaras otro método de clustering?
- **Comparación:** ¿Coincide la respuesta del LLM con lo que observaste en tu propio análisis?
- **Sugerencias:** ¿Qué recomendarías para mejorar la segmentación?

Reflexión Final de la Clase

- ¿PCA fue útil para reducir la dimensionalidad sin perder información clave?
- ¿Qué diferencias notaste entre clustering con datos originales y después de PCA?
- ¿Cómo puede una IA ayudar en el análisis de datos en el futuro?

Bibliografía

1. Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques*. Elsevier.
2. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning: With Applications in R*. Springer.
3. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer.
4. McKinney, W. (2017). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O'Reilly Media.
5. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
6. Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and Statistical Modeling with Python. *SciPy Conference*.
7. Scikit-learn Developers. (n.d.). *Scikit-learn: Machine Learning in Python*. Recuperado de <https://scikit-learn.org/>
8. Seaborn Documentation. (n.d.). *Seaborn: Statistical Data Visualization*. Recuperado de <https://seaborn.pydata.org/>

4. Visualizacion de Datos.html

Clase 4: Visualización de Datos

1. PRESENTACIÓN

Objetivo de la Clase

La visualización de datos es una herramienta fundamental en la minería de datos. No solo nos ayuda a entender los datos, sino que también facilita la comunicación de hallazgos y patrones a otras personas.

En esta clase, aprenderemos a:

- Identificar cuáles son los tipos de gráficos más adecuados para cada tipo de análisis.
- Usar Matplotlib y Seaborn para crear visualizaciones efectivas en Python.
- Aplicar técnicas avanzadas como pairplots y heatmaps para explorar relaciones entre múltiples variables.
- Interpretar gráficos correctamente para tomar mejores decisiones.

Importancia de la Visualización de Datos

- Permite comprender rápidamente grandes volúmenes de datos.
- Ayuda a detectar valores atípicos y errores en los datos.
- Facilita la identificación de patrones y tendencias.
- Mejora la presentación y comunicación de análisis a audiencias técnicas y no técnicas.

Ejemplo real: Un equipo de científicos de datos está analizando el comportamiento de clientes en una tienda en línea. A través de gráficos de dispersión y heatmaps, pueden visualizar qué productos suelen comprarse juntos y optimizar las recomendaciones.

1.1. ¿Por qué es importante la visualización de datos?

La visualización de datos es un paso clave en cualquier proceso de minería de datos. Permite comprender mejor los datos, detectar patrones, identificar valores atípicos y comunicar hallazgos de manera clara.

Beneficios de la visualización de datos:

- Facilita la comprensión de grandes volúmenes de información.
- Ayuda a detectar errores y valores atípicos en los datos.
- Permite identificar relaciones entre variables.
- Mejora la comunicación de resultados en equipos multidisciplinares.

Ejemplo real: Un banco analiza datos de clientes para otorgar créditos. Un gráfico de dispersión entre ingresos y nivel de endeudamiento puede ayudar a visualizar patrones y tomar mejores decisiones.

1.2. Tipos de Visualización y Cuándo Usarlos

Cada tipo de gráfico tiene un propósito específico. La siguiente tabla resume qué gráfico usar según el análisis que queremos realizar.

Tipo de Análisis	Objetivo	Gráfico Recomendado
Distribución de una variable	Entender cómo se distribuyen los valores.	Histograma, KDE Plot
Valores atípicos en una variable	Detectar outliers o datos anómalos.	Boxplot
Relación entre dos variables numéricas	Ver si hay correlación o dependencia.	Scatter Plot, Line Plot
Comparación de categorías	Comparar valores de diferentes grupos.	Bar Chart, Violin Plot
Evolución de datos en el tiempo	Identificar tendencias en series temporales.	Line Plot
Relación entre múltiples variables	Ver interacciones en varios ejes.	Pairplot, Heatmap

Ejemplo en Python – Diferentes Tipos de Visualización

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Crear datos ficticios
df = pd.DataFrame({
    "Edad":      [25, 32, 40, 35, 50, 45, 29, 37, 41, 60],
    "Ingresos":  [30000, 45000, 60000, 55000, 80000, 75000, 40000, 58000, 72000, 90000],
    "Categoría": ["Joven", "Adulto", "Adulto", "Adulto", "Senior", "Senior", "Joven", "Adulto", "Adulto", "Senior"]
})

# Histograma de edades
sns.histplot(df["Edad"], bins=5, kde=True)
plt.title("Distribución de Edades")
plt.show()

# Boxplot de ingresos
sns.boxplot(x=df["Ingresos"])
plt.title("Boxplot de Ingresos")
plt.show()

# Scatter Plot entre Edad e Ingresos
sns.scatterplot(x=df["Edad"], y=df["Ingresos"], hue=df["Categoría"])
plt.title("Relación entre Edad e Ingresos")
plt.show()
```

Conclusión:

- El histograma nos permite ver la distribución de edades.
- El boxplot ayuda a detectar posibles valores atípicos en ingresos.
- El scatter plot muestra la relación entre Edad e Ingresos, diferenciando categorías.

Ejercicio para la Sección 1

Objetivo: Practicar la selección de gráficos según el tipo de análisis.

Instrucciones:

1. Selecciona el gráfico adecuado para cada uno de los siguientes análisis:
 - Comparar el salario promedio en diferentes industrias.
 - Analizar si existe una relación entre la cantidad de horas trabajadas y el nivel de productividad.
 - Detectar valores atípicos en los montos de transacciones bancarias.
 - Observar la distribución de notas de un curso.

2. Explica por qué elegiste cada tipo de gráfico.
3. Si tienes conocimientos en Python, escribe un código utilizando Seaborn para visualizar alguno de los casos.

Sección 2: Matplotlib y Seaborn

2.1. ¿Qué es Matplotlib y cómo usarlo?

Matplotlib es la biblioteca más básica y flexible para crear gráficos en Python. Se usa principalmente para gráficos simples como líneas, barras y dispersión.

Ejemplo en Python – Crear un gráfico de líneas con Matplotlib

```
import matplotlib.pyplot as plt

# Datos de ejemplo
x = [1, 2, 3, 4, 5]
y = [10, 20, 25, 30, 50]

# Crear gráfico de líneas
plt.plot(x, y, marker='o', linestyle='-', color='b', label="Crecimiento")
plt.xlabel("Tiempo")
plt.ylabel("Valor")
plt.title("Ejemplo de Gráfico de Líneas")
plt.legend()
plt.show()
```

Conclusión: Matplotlib permite personalizar cada aspecto del gráfico, como colores, etiquetas y estilos de líneas.

2.2. ¿Qué es Seaborn y cómo usarlo?

Seaborn está basado en Matplotlib, pero ofrece una sintaxis más sencilla y gráficos más atractivos por defecto. Se usa para visualizaciones estadísticas avanzadas, como distribuciones de datos, correlaciones y visualización de múltiples variables.

Ejemplo en Python – Histogramas y Boxplots con Seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Crear un DataFrame con datos ficticios
df = pd.DataFrame({
    "Edad": [25, 32, 40, 35, 50, 45, 29, 37, 41, 60],
    "Ingresos": [30000, 45000, 60000, 55000, 80000, 75000, 40000, 58000, 72000, 90000]
})

# Histograma de edades
sns.histplot(df["Edad"], bins=5, kde=True)
plt.title("Distribución de Edades")
plt.show()

# Boxplot de ingresos
sns.boxplot(x=df["Ingresos"])
plt.title("Boxplot de Ingresos")
plt.show()
```

Conclusión: Seaborn es ideal para gráficos estadísticos más complejos, como histogramas con KDE, boxplots y pairplots.

2.3. Comparación entre Matplotlib y Seaborn

Característica	Matplotlib	Seaborn
Facilidad de uso	Más flexible pero requiere más código.	Más intuitivo y automático.
Personalización	Muy alta (permite modificar cada detalle).	Alta, pero con menos código necesario.
Aplicaciones	Gráficos básicos y personalizables.	Gráficos estadísticos más avanzados.
Ejemplo típico	Gráficos de líneas y barras.	Histogramas, heatmaps y pairplots.

¿Cuándo usar cada uno?

- Usa **Matplotlib** cuando necesites total control sobre los detalles del gráfico.
- Usa **Seaborn** cuando necesites gráficos estadísticos listos para usar con menos código.

Ejercicio para la Sección 2

Objetivo: Practicar el uso de Matplotlib y Seaborn creando gráficos de distintos tipos.

Instrucciones:

1. Usa Matplotlib para crear un gráfico de barras que muestre el promedio de ingresos de diferentes categorías de empleados.
2. Usa Seaborn para crear un scatterplot que muestre la relación entre Edad e Ingresos en un dataset de empleados.
3. Explica cuál de los dos gráficos es más fácil de interpretar y por qué.

Pista para la solución en Python

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Crear DataFrame
df = pd.DataFrame({
    "Categoría":      ["Junior", "Semi-Senior", "Senior"],
    "Salario_Promedio": [35000, 60000, 90000]
})

# Gráfico de barras con Matplotlib
plt.bar(df["Categoría"], df["Salario_Promedio"], color=["blue", "green", "red"])
plt.xlabel("Categoría de Empleado")
plt.ylabel("Salario Promedio")
plt.title("Salario Promedio por Categoría")
plt.show()

# Crear DataFrame con edades e ingresos
df2 = pd.DataFrame({
    "Edad":      [25, 32, 40, 35, 50, 45, 29, 37, 41, 60],
    "Ingresos": [30000, 45000, 60000, 55000, 80000, 75000, 40000, 58000, 72000, 90000]
})

# Scatterplot con Seaborn
sns.scatterplot(x=df2["Edad"], y=df2["Ingresos"])
plt.title("Relación entre Edad e Ingresos")
plt.show()
```

Preguntas de Reflexión

1. ¿Cuál de los dos gráficos te parece más útil para analizar la información?
2. ¿Cómo podría mejorarse la interpretación de estos gráficos?
3. ¿Cómo podrías aplicar esto en un dataset real?

Sección 3: Gráficos Avanzados con Seaborn (Pairplot, Heatmaps y Violin Plots)

Objetivo: Aprender a utilizar gráficos avanzados en Seaborn para visualizar relaciones complejas en los datos.

3.1. Pairplot: Exploración Multivariable

¿Qué es un Pairplot?

Es una herramienta que muestra relaciones entre múltiples variables en un dataset de manera eficiente.

¿Cuándo usarlo?

- Cuando queremos ver cómo se relacionan múltiples variables numéricas en un solo gráfico.
- Cuando queremos detectar correlaciones visualmente sin calcular matrices de correlación.

Ejemplo en Python – Pairplot con Seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Cargar dataset de pingüinos de Seaborn
df = sns.load_dataset("penguins")

# Crear Pairplot
sns.pairplot(df, hue="species", diag_kind="kde")
plt.show()
```

Conclusión:

- Cada gráfico en la diagonal muestra la distribución univariada de cada variable.
- Los gráficos fuera de la diagonal muestran relaciones entre pares de variables.

3.2. Heatmaps: Mapas de Correlación

¿Qué es un Heatmap?

Es un mapa de calor que muestra cómo se correlacionan las variables numéricas en un dataset.

¿Cuándo usarlo?

- Cuando queremos analizar fuerza y dirección de correlación entre variables.
- Cuando buscamos variables redundantes que pueden eliminarse o agruparse.

Ejemplo en Python – Heatmap de correlaciones

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Cargar dataset
df = sns.load_dataset("penguins").dropna()

# Calcular matriz de correlación
correlaciones = df.corr(numeric_only=True)

# Crear Heatmap
sns.heatmap(correlaciones, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Mapa de Correlación entre Variables")
plt.show()
```

Conclusión:

- Un coeficiente cercano a **1** indica alta correlación positiva (ejemplo: peso y tamaño).
- Un coeficiente cercano a **-1** indica alta correlación negativa (ejemplo: velocidad y tiempo de viaje).
- Un coeficiente cercano a **0** significa que las variables no están relacionadas.

3.3. Violin Plots: Comparación de Distribuciones

¿Qué es un Violin Plot?

Es una mezcla entre un boxplot y un histograma, que muestra la distribución de los datos dentro de cada categoría.

¿Cuándo usarlo?

- Cuando queremos ver cómo varían los valores en diferentes grupos.
- Cuando queremos detectar asimetrías en la distribución de datos.

Ejemplo en Python – Violin Plot

```
import seaborn as sns
import matplotlib.pyplot as plt

# Cargar dataset
df = sns.load_dataset("penguins")

# Crear Violin Plot
sns.violinplot(x="species", y="bill_length_mm", data=df, palette="muted")
plt.title("Distribución del Largo del Pico por Especie")
plt.show()
```

Conclusión:

- El ancho del violín representa dónde se encuentran más datos.
- Las colas indican valores extremos o atípicos.

Ejercicio para la Sección 3

Objetivo: Aplicar gráficos avanzados para extraer insights de un dataset.

Instrucciones:

1. Carga el dataset de diamantes de Seaborn (**diamonds**).
2. Crea un Pairplot para analizar la relación entre precio, quilates y profundidad.
3. Genera un Heatmap para visualizar la correlación entre las variables numéricas.
4. Crea un Violin Plot para comparar la distribución del precio en cada categoría de corte (**cut**).
5. Escribe una conclusión basada en los gráficos.

Pista para la solución en Python

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Cargar dataset
df = sns.load_dataset("diamonds")

# 1. Pairplot entre precio, quilates y profundidad
sns.pairplot(df[["price", "carat", "depth"]])
plt.show()

# 2. Heatmap de correlaciones
correlaciones = df.corr(numeric_only=True)
sns.heatmap(correlaciones, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Mapa de Correlación de Diamantes")
plt.show()

# 3. Violin Plot para comparar precio según corte
sns.violinplot(x="cut", y="price", data=df, palette="muted")
plt.title("Distribución del Precio por Tipo de Corte")
plt.show()
```

Preguntas de Reflexión

1. ¿Qué variables están más correlacionadas con el precio?
2. ¿Cómo varía la distribución del precio en los diferentes tipos de corte?
3. ¿Qué otros insights puedes extraer de estos gráficos?

Actividad Obligatoria

Entregarla en el espacio para la misma dentro del Campus.

Objetivo: Aplicar todas las técnicas de visualización de datos vistas en la clase para analizar un dataset real y compartir los hallazgos en el foro.

4.1. Ejercicio Integrador

Supongamos que estamos analizando un conjunto de datos de ventas de una tienda en línea. Queremos entender el comportamiento de los clientes y visualizar patrones clave que puedan ayudar a mejorar la estrategia de ventas.

Dataset para el ejercicio

Cliente	Edad	Ingresos	Compras	Categoría
1	25	50000	5	A
2	40	60000	15	B
3	35	70000	10	C
4	50	80000	20	A
5	30	45000	7	B
6	45	85000	18	C
7	38	72000	12	A
8	29	48000	6	B

Tareas a realizar

- 1. **Análisis Exploratorio de Datos (EDA)**
 - Generar histogramas para Edad e Ingresos.
 - Crear un boxplot para visualizar valores atípicos en Compras.
 - Generar un scatter plot para analizar la relación entre Ingresos y Compras.
- 2. **Visualización Avanzada**
 - Crear un Pairplot con las variables numéricas.
 - Generar un Heatmap de correlaciones entre todas las variables numéricas.
 - Utilizar un Violin Plot para comparar la distribución de Compras según la Categoría.
- 3. **Interacción con una Herramienta de IA (LLM)**
 - Hacer una consulta a ChatGPT, Gemini u otra herramienta LLM para obtener insights sobre los datos.
 - Ejemplo de consulta: *"Tengo un dataset con datos de clientes (edad, ingresos y número de compras). ¿Qué visualizaciones me recomendarías para encontrar patrones?"*
 - Comparar la respuesta con lo aprendido en la clase y reflexionar sobre su utilidad.

Código Base para la Solución

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Crear DataFrame
df = pd.DataFrame({
    "Cliente": [1, 2, 3, 4, 5, 6, 7, 8],
    "Edad": [25, 40, 35, 50, 30, 45, 38, 29],
    "Ingresos": [50000, 60000, 70000, 80000, 45000, 85000, 72000, 48000],
    "Compras": [5, 15, 10, 20, 7, 18, 12, 6],
    "Categoría": ["A", "B", "C", "A", "B", "C", "A", "B"]
})

# 1) Análisis Exploratorio
sns.histplot(df["Edad"], bins=5, kde=True)
plt.title("Distribución de Edades")
plt.show()

sns.boxplot(x=df["Compras"])
plt.title("Boxplot de Compras")
plt.show()

sns.scatterplot(x=df["Ingresos"], y=df["Compras"])
plt.title("Relación entre Ingresos y Compras")
plt.show()

# 2) Visualización Avanzada
sns.pairplot(df[["Edad", "Ingresos", "Compras"]])
plt.show()

# Mapa de calor de correlaciones
correlaciones = df.corr(numeric_only=True)
sns.heatmap(correlaciones, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Mapa de Correlación de Variables")
plt.show()

# Violin Plot de Compras por Categoría
sns.violinplot(x="Categoría", y="Compras", data=df, palette="muted")
plt.title("Distribución de Compras por Categoría")
plt.show()
```

Bibliografía

1. Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques* (3rd ed.). Elsevier.
2. Wickham, H., & Grolemund, G. (2016). *R for Data Science*. O'Reilly Media.
3. McKinney, W. (2017). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O'Reilly Media.
4. Seaborn Documentation. (n.d.). *Seaborn: Statistical Data Visualization*. Recuperado de <https://seaborn.pydata.org/>
5. Matplotlib Developers. (n.d.). *Matplotlib: Visualization with Python*. Recuperado de <https://matplotlib.org/>
6. Tufte, E. R. (2001). *The Visual Display of Quantitative Information*. Graphics Press.
7. ChatGPT & Gemini (2023). *Uso de Modelos de Lenguaje para la Exploración de Datos*. Recuperado de <https://openai.com>

5. Limpieza y Transformacion Avanzada

Clase N° 5: Limpieza y Transformación de Datos en Minería de Datos

1. PRESENTACIÓN

Objetivo de la Clase

El preprocesamiento de datos es una fase crítica en la minería de datos. Antes de aplicar modelos de Machine Learning, debemos asegurarnos de que los datos sean limpios, consistentes y bien estructurados.

En esta clase, aprenderemos a:

- Detectar y manejar datos faltantes usando estrategias como eliminación e imputación.
- Identificar y tratar outliers con métodos estadísticos.
- Transformar datos mediante normalización, estandarización y codificación de variables categóricas.
- Usar Python y bibliotecas como Pandas y Scikit-learn para preprocesar datos de manera eficiente.

Importancia del Preprocesamiento de Datos

- Datos sucios pueden generar errores en los análisis y modelos predictivos.
- La transformación correcta de los datos mejora la precisión de los modelos de Machine Learning.
- Algunas técnicas como normalización y estandarización ayudan a que los modelos converjan más rápido y funcionen mejor.

Ejemplo real: Un hospital quiere predecir el tiempo de recuperación de pacientes basándose en su edad, tipo de tratamiento y otros factores. Si los datos tienen valores faltantes en edad o tipo de tratamiento, los resultados serán poco confiables o sesgados.

Sección 1: Manejo de Datos Faltantes

¿Por qué aparecen valores faltantes en los datos?

Los datos faltantes pueden deberse a varias razones:

- **Errores de entrada:** Un usuario no completó un campo en un formulario.
- **Fallas en la recolección de datos:** Sensores que fallan al registrar información.
- **Filtros o reglas de negocio:** Algunas variables solo se registran bajo ciertas condiciones.

1.1. Estrategias para manejar datos faltantes

Estrategia	Descripción	Cuándo Usarla	Ventajas	Desventajas
Eliminar filas/ columnas	Descarta datos incompletos.	Cuando los valores faltantes son pocos y aleatorios.	Fácil de aplicar.	Puede eliminar información útil.
Imputación con media/mediana/ moda	Rellena valores faltantes con la media, mediana o moda de la columna.	Cuando los valores faltantes son pocos y los datos siguen una distribución estable.	Preserva la cantidad de datos.	Puede sesgar los resultados si hay muchos valores faltantes.
Imputación con valores específicos	Sustituye valores faltantes con un valor arbitrario (ej. "Desconocido").	Cuando queremos evitar perder datos sin afectar la distribución.	Mantiene la estructura del dataset.	Puede introducir información no realista.
Modelos de imputación (KNN, regresión, MICE)	Predice los valores faltantes basándose en otras variables.	Cuando hay un patrón predecible en los datos.	Mantiene consistencia en los datos.	Requiere mayor computación y validación.

1.2. Ejemplo en Python – Manejo de Datos Faltantes

```
import pandas as pd
import numpy as np

# Crear DataFrame con valores faltantes
df = pd.DataFrame({
    "Nombre": ["Ana", "Juan", "Pedro", "María", "Luis"],
    "Edad": [25, np.nan, 40, np.nan, 30],
    "Salario": [50000, 60000, np.nan, 55000, 45000]
})
print("Datos Originales:
", df)

# Imputación con la mediana
df["Edad"].fillna(df["Edad"].median(), inplace=True)

# Eliminación de filas con valores nulos
df.dropna(subset=["Salario"], inplace=True)
print("
Datos después del Preprocesamiento:
", df)
```

Conclusión:

- La imputación con la mediana preserva la información de la variable Edad.
- Eliminar filas con valores nulos en Salario evita introducir datos incorrectos.

Ejercicio para la Sección 1

Objetivo: Aplicar técnicas de manejo de valores faltantes en un dataset real.

Instrucciones:

1. Carga el dataset Titanic de Seaborn (`sns.load_dataset("titanic")`).
2. Identifica qué columnas tienen valores nulos y cuántos valores faltantes hay en cada una.
3. Elimina las filas con valores nulos en la columna `age`.
4. Imputa los valores faltantes en la columna `embark_town` usando la moda.
5. Explica por qué elegiste esas estrategias y qué impacto pueden tener en un análisis posterior.

Pista para la solución en Python

```
import seaborn as sns
import pandas as pd

# Cargar dataset Titanic
df = sns.load_dataset("titanic")

# Ver cantidad de valores nulos por columna
print("Valores nulos por columna:
", df.isnull().sum())

# 1. Eliminar filas con valores nulos en 'age'
df_cleaned = df.dropna(subset=["age"])

# 2. Imputar valores nulos en 'embark_town' con la moda
df_cleaned["embark_town"].fillna(df_cleaned["embark_town"].mode()[0], inplace=True)

print("
Valores nulos después del preprocesamiento:
", df_cleaned.isnull().sum())
```

Preguntas de Reflexión

1. ¿Cómo afecta la eliminación de datos a la cantidad de información disponible?
2. ¿Qué problemas pueden surgir si imputamos valores en variables categóricas?
3. ¿Cómo podríamos validar que nuestras imputaciones no introducen sesgos?

Sección 2: Detección y Tratamiento de Outliers

Objetivo: Identificar valores atípicos en un conjunto de datos y aplicar técnicas adecuadas para manejarlos.

2.1. ¿Qué son los Outliers y por qué Importan?

- Un **outlier** (valor atípico) es un punto de datos que se aleja significativamente del resto de las observaciones.
- Los outliers pueden deberse a errores de medición, errores de entrada o fenómenos reales poco comunes.

Ejemplo de un outlier en la vida real: Si analizamos los salarios de empleados en una empresa, pero encontramos que un solo empleado gana 100 veces más que los demás, ese dato podría ser un error o una excepción justificada (ejemplo: CEO de la empresa).

2.2. Métodos para Detectar Outliers

Método	Descripción	Cuándo Usarlo
Boxplot	Identifica valores atípicos fuera del rango intercuartil.	Datos numéricos con distribución normal.
Z-score	Detecta outliers midiendo la desviación estándar desde la media.	Datos con distribución normal.

Método	Descripción	Cuándo Usarlo
IQR (Rango Intercuartil)	Considera valores fuera de 1.5 veces el rango intercuartil como outliers.	Datos con distribución sesgada o no normal.
DBSCAN	Detecta outliers en datos espaciales y de clustering.	Cuando los outliers son puntos alejados en múltiples dimensiones.

2.3. Ejemplo en Python – Detección de Outliers con Boxplot y Z-score

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import zscore

# Crear DataFrame con un outlier
df = pd.DataFrame({
    "Empleado": ["A", "B", "C", "D", "E", "F", "G"],
    "Salario": [50000, 52000, 54000, 58000, 60000, 62000, 250000] # 250000 es un outlier
})

# Boxplot para visualizar outliers
sns.boxplot(x=df["Salario"])
plt.title("Boxplot de Salarios")
plt.show()

# Detectar outliers con Z-score
df["Z_Score"] = zscore(df["Salario"])
outliers = df[df["Z_Score"].abs() > 3]
print("Outliers detectados: ", outliers)
```

Conclusión:

- Los boxplots ayudan a visualizar valores extremos de forma gráfica.
- El Z-score nos permite identificar valores que se alejan más de 3 desviaciones estándar de la media.

2.4. Estrategias para Manejar Outliers

Estrategia	Descripción	Cuándo Usarla
Eliminar el outlier	Se descarta el dato si se confirma que es un error.	Cuando el outlier es un error evidente.
Transformación logarítmica	Se usa logaritmo para reducir el impacto de valores extremos.	Cuando los outliers afectan la distribución de datos.
Winsorización	Sustituye valores extremos con percentiles más cercanos.	Cuando queremos reducir el impacto sin eliminar datos.
Modelos robustos	Algunos algoritmos (ej. regresión robusta) pueden manejar outliers sin eliminarlos.	Cuando los outliers son parte del fenómeno real.

Ejercicio para la Sección 2

Objetivo: Aplicar detección y tratamiento de outliers en un dataset real.

Instrucciones:

1. Carga el dataset Diamonds de Seaborn (`sns.load_dataset("diamonds")`).
2. Crea un boxplot para visualizar outliers en la columna `price`.
3. Aplica el método IQR para detectar y eliminar valores atípicos en `price`.
4. Explica qué impacto tendría eliminar estos valores en un modelo de Machine Learning.

Pista para la solución en Python

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Cargar dataset
df = sns.load_dataset("diamonds")

# Boxplot de precios
sns.boxplot(x=df["price"])
plt.title("Boxplot de Precios de Diamantes")
plt.show()

# Cálculo del IQR
Q1 = df["price"].quantile(0.25)
Q3 = df["price"].quantile(0.75)
IQR = Q3 - Q1

# Definir límites para detectar outliers
limite_inferior = Q1 - 1.5 * IQR
limite_superior = Q3 + 1.5 * IQR

# Filtrar valores dentro de los límites
df_cleaned = df[(df["price"] >= limite_inferior) & (df["price"] <= limite_superior)]
print("Antes:", len(df), "Después:", len(df_cleaned))
```

Preguntas de Reflexión

1. ¿Cómo cambia la distribución de **price** después de eliminar outliers?
2. ¿Eliminar outliers mejora siempre un análisis? ¿Cuándo puede ser contraproducente?
3. ¿Existen casos en los que los outliers pueden ser la parte más importante de los datos?

Sección 3: Normalización, Estandarización y Codificación de Datos Categóricos

Objetivo: Aplicar técnicas de transformación de datos numéricos y categóricos para mejorar la calidad de los modelos de minería de datos.

3.1. Normalización y Estandarización

¿Por qué transformar los datos?

Algunos algoritmos de Machine Learning son sensibles a la escala de los datos. Si una variable tiene valores muy grandes en comparación con otra, puede dominar el modelo e influenciar los resultados.

Diferencias entre Normalización y Estandarización

Técnica	Fórmula	Rango de salida	Cuándo usarla
Normalización (Min-Max Scaling)	$X_{norm} = \frac{(X - X_{min})}{(X_{max} - X_{min})}$	[0, 1]	Cuando los datos no siguen una distribución normal.
Estandarización (Z-score Scaling)	$X_{std} = \frac{(X - \mu)}{\sigma}$	Centrado en 0 con desviación estándar 1	Cuando los datos tienen una distribución normal.

Ejemplo en Python – Aplicando Normalización y Estandarización

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Crear DataFrame con datos ficticios
df = pd.DataFrame({
    "Edad": [18, 25, 30, 35, 40, 50, 60, 70],
    "Salario": [25000, 32000, 40000, 50000, 60000, 75000, 90000, 120000]
})

# Normalización (Min-Max Scaling)
scaler_minmax = MinMaxScaler()
df["Salario_Normalizado"] = scaler_minmax.fit_transform(df[["Salario"]])

# Estandarización (Z-score)
scaler_standard = StandardScaler()
df["Salario_Estandarizado"] = scaler_standard.fit_transform(df[["Salario"]])

print("
Datos transformados:
", df)
```

Conclusión:

- La normalización ajusta los datos al rango [0,1], útil para modelos como redes neuronales.
- La estandarización transforma los datos con media 0 y desviación estándar 1, ideal para algoritmos sensibles a la distribución normal como regresión logística.

3.2. Codificación de Variables Categóricas

¿Por qué transformar variables categóricas?

Los algoritmos de Machine Learning no pueden interpretar directamente datos categóricos. Debemos convertir estas variables a valores numéricos.

Métodos de Codificación

Técnica	Descripción	Ejemplo
Label Encoding	Asigna un número único a cada categoría.	{"Rojo": 0, "Verde": 1, "Azul": 2}
One-Hot Encoding	Crea una columna binaria para cada categoría.	{"Rojo": [1,0,0], "Verde": [0,1,0], "Azul": [0,0,1]}
Ordinal Encoding	Asigna valores según un orden lógico.	{"Bajo": 1, "Medio": 2, "Alto": 3}

Ejemplo en Python – Label Encoding y One-Hot Encoding

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Crear DataFrame con variables categóricas
df = pd.DataFrame({"Color": ["Rojo", "Verde", "Azul", "Verde", "Rojo"]})

# Aplicar Label Encoding
le = LabelEncoder()
df["Color_Label"] = le.fit_transform(df["Color"])

# Aplicar One-Hot Encoding
df_ohe = pd.get_dummies(df["Color"], prefix="Color")

print("
Label Encoding:
", df)
print("
One-Hot Encoding:
", df_ohe)
```

Conclusión:

- Label Encoding es útil para modelos de árboles de decisión, pero introduce un orden que puede no ser real.
- One-Hot Encoding es mejor para modelos lineales, pero aumenta la dimensionalidad del dataset.

Ejercicio para la Sección 3

Objetivo: Aplicar transformación de datos numéricos y categóricos en un dataset real.

Instrucciones:

1. Carga el dataset Titanic de Seaborn (`sns.load_dataset("titanic")`).
2. Aplica normalización Min-Max en la columna `fare`.
3. Convierte la columna `sex` a valores numéricos usando One-Hot Encoding.
4. Explica qué impacto puede tener esta transformación en un modelo de clasificación.

Pista para la solución en Python

```
import seaborn as sns
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Cargar dataset Titanic
df = sns.load_dataset("titanic")

# Normalización de la columna fare
scaler = MinMaxScaler()
df["fare_normalizado"] = scaler.fit_transform(df[["fare"]])

# One-Hot Encoding de la columna sex
df = pd.get_dummies(df, columns=["sex"], prefix="Sexo")

print("
Datos transformados:
", df[["fare", "fare_normalizado", "Sexo_female", "Sexo_male"]].head())
```

Preguntas de Reflexión

1. ¿Por qué One-Hot Encoding es mejor que Label Encoding en este caso?
2. ¿Cómo cambiaría el rendimiento de un modelo si usamos datos sin normalizar?
3. ¿Cuándo puede ser preferible usar una codificación ordinal?

Sección 4: Ejercicio Integrador y Publicación en el Foro

Objetivo: Aplicar todas las técnicas de limpieza y transformación de datos vistas en la clase a un dataset real y fomentar la discusión en el foro.

4.1. Ejercicio Integrador

Supongamos que trabajamos en una empresa de análisis financiero y tenemos un dataset con información de clientes bancarios. Nuestro objetivo es preparar los datos para un modelo de predicción de solvencia financiera.

Dataset para el ejercicio

Cliente	Edad	Ingresos	Historial de Pagos	Categoría
1	25	50000	Bueno	A
2	40	60000	Regular	B
3	35	70000	Malo	C
4	50	80000	Bueno	A
5	30	45000	Regular	B
6	45	85000	Malo	C
7	38	72000	Bueno	A
8	29	48000	Regular	B

Tareas a realizar

1. Limpieza de Datos

- Identificar y manejar valores nulos en Edad o Ingresos.
- Detectar outliers en Ingresos usando el método IQR y decidir si eliminarlos o transformarlos.

2. Transformación de Datos Numéricos

- Normalizar Ingresos usando Min-Max Scaling.
- Estandarizar Edad con Z-score Scaling.

3. Transformación de Datos Categóricos

- Aplicar One-Hot Encoding a la columna Historial de Pagos.
- Aplicar Label Encoding a la columna Categoría.

4. Interacción con una Herramienta de IA (LLM)

- Hacer una consulta a ChatGPT, Gemini u otra herramienta LLM sobre técnicas avanzadas de preprocesamiento de datos.
- Ejemplo de consulta: *"¿Qué métodos avanzados de imputación de datos recomiendas para conjuntos de datos con muchas variables numéricas y categóricas?"*
- Comparar la respuesta con lo aprendido en la clase y reflexionar sobre su utilidad.

Código Base para la Solución

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler, LabelEncoder

# Crear DataFrame
df = pd.DataFrame({
    "Cliente":      [1, 2, 3, 4, 5, 6, 7, 8],
    "Edad":         [25, 40, 35, 50, 30, 45, 38, 29],
    "Ingresos":     [50000, 60000, 70000, 80000, 45000, 85000, 72000, 48000],
    "Historial_Pagos": ["Bueno", "Regular", "Malo", "Bueno", "Regular", "Malo", "Bueno", "Regular"],
    "Categoría":     ["A", "B", "C", "A", "B", "C", "A", "B"]
})

# 1) Manejo de valores nulos
df.fillna(df.median(numeric_only=True), inplace=True)

# 2) Detección de outliers con IQR
Q1 = df["Ingresos"].quantile(0.25)
Q3 = df["Ingresos"].quantile(0.75)
IQR = Q3 - Q1
df_cleaned = df[(df["Ingresos"] >= Q1 - 1.5*IQR) & (df["Ingresos"] <= Q3 + 1.5*IQR)]

# 3) Transformación numérica
scaler_minmax = MinMaxScaler()
df_cleaned["Ingresos_Normalizado"] = scaler_minmax.fit_transform(df_cleaned[["Ingresos"]])

scaler_std = StandardScaler()
df_cleaned["Edad_Estandarizada"] = scaler_std.fit_transform(df_cleaned[["Edad"]])

# 4) Transformación categórica
df_cleaned = pd.get_dummies(df_cleaned, columns=["Historial_Pagos"], prefix="Historial")
le = LabelEncoder()
df_cleaned["Categoría_Label"] = le.fit_transform(df_cleaned["Categoría"])

print("
Datos transformados:
", df_cleaned)
```

Instrucciones para compartir resultados

Publica un mensaje en el foro obligatorio de la clase con los siguientes puntos:

- Un resumen del proceso de limpieza y transformación de datos.
- Capturas o gráficos que muestren antes y después de la limpieza y transformación.
- ¿Cómo afectó la normalización y estandarización a los datos?
- ¿Qué ventajas y desventajas observaste en el uso de One-Hot Encoding vs. Label Encoding?
- ¿Qué insights obtuviste usando una herramienta LLM (ChatGPT, Gemini, etc.)?

Ejemplo de Publicación en el Foro

Resumen del Preprocesamiento: "Se identificaron y eliminaron outliers en la columna de ingresos. Luego, se aplicó normalización Min-Max en Ingresos y estandarización Z-score en Edad. Finalmente, se usó One-Hot Encoding para Historial de Pagos y Label Encoding para Categoría."

Resultados Visuales: "Antes de la transformación, los ingresos tenían una gran variabilidad. Después de la normalización, todos los valores se encuentran entre 0 y 1, lo que facilitará la convergencia de modelos de Machine Learning."

Consulta a ChatGPT: "Pregunté a ChatGPT sobre imputación avanzada de valores nulos y recomendó probar el método KNN Imputer. Implementé esta estrategia y observé que la predicción es más precisa cuando las variables tienen correlación significativa."

4.3. Participación en la Publicación de un Compañero

Comenta en la publicación de otro estudiante:

- **Pregunta:** ¿Cómo cambiarían los resultados si usáramos otro método de imputación de valores nulos?
- **Comparación:** ¿Coincide la respuesta del LLM con lo que observaste en tu propio análisis?
- **Sugerencias:** ¿Cómo podríamos mejorar la transformación de datos?

Reflexión Final de la Clase

- ¿Cómo impactó la limpieza y transformación de datos en la calidad del dataset?
- ¿Qué técnicas de preprocesamiento resultaron más efectivas?
- ¿Qué problemas pueden surgir si los datos no se limpian correctamente?

Bibliografía

1. Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques* (3rd ed.). Elsevier.
2. McKinney, W. (2017). *Python for Data Analysis*. O'Reilly Media.
3. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
4. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*. Springer.
5. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.
6. Wickham, H., & Grolemond, G. (2016). *R for Data Science*. O'Reilly Media.
7. Provost, F., & Fawcett, T. (2013). *Data Science for Business*. O'Reilly Media.

6. Clustering

Clase N° 6: Clustering en Minería de Datos

Introducción

Objetivo de la Clase

El clustering es una de las técnicas más utilizadas en minería de datos para descubrir patrones ocultos en grandes volúmenes de información. A diferencia del aprendizaje supervisado, el clustering es una técnica de **aprendizaje no supervisado**, lo que significa que no se cuenta con etiquetas previas para los datos.

En esta clase, aprenderemos a:

- Entender qué es el clustering y cuándo se aplica.
- Explorar diferentes algoritmos de clustering como K-Means, DBScan y Mapas Autoorganizados (SOM).
- Comparar los distintos enfoques y evaluar su desempeño.
- Implementar algoritmos de clustering en Python con Scikit-learn y otras bibliotecas.
- Analizar resultados y seleccionar el mejor modelo para un conjunto de datos.

Importancia del Clustering en Minería de Datos

- Segmentación de clientes en marketing para mejorar estrategias de ventas.
- Agrupación de documentos y textos en sistemas de recomendación.
- Detección de anomalías en transacciones bancarias o ciberseguridad.

- Clasificación automática de imágenes y datos biomédicos.

Ejemplo real: Un banco desea identificar diferentes perfiles de clientes según su comportamiento financiero. Mediante clustering, puede agrupar a los clientes en segmentos similares (clientes con bajo riesgo, clientes con alto consumo de crédito, etc.), lo que permitirá diseñar estrategias personalizadas.

Sección 1: Conceptos Fundamentales del Clustering

¿Qué es el Clustering?

El clustering es un proceso de agrupamiento de datos en subconjuntos homogéneos, de manera que los datos dentro de un grupo son más similares entre sí que con los datos de otros grupos.

1.1. Características del Clustering

- **No supervisado:** No requiere etiquetas previas en los datos.
- Identifica patrones ocultos en los datos sin intervención humana.
- Los resultados dependen del algoritmo elegido y de los parámetros utilizados.

1.2. Tipos de Clustering

Tipo de Clustering	Descripción	Ejemplo de Uso
Particional	Divide los datos en un número fijo de grupos.	K-Means, K-Medoids
Jerárquico	Crea una estructura de árbol para los clusters.	Dendrogramas en biología
Basado en densidad	Agrupar datos en regiones de alta densidad.	DBScan para detección de anomalías
Basado en redes neuronales	Utiliza modelos autoorganizados.	SOM en análisis de imágenes

¿Cómo elegir un tipo de clustering?

- Si se conoce el número de grupos → **K-Means o K-Medoids**.
- Si hay ruido o valores atípicos en los datos → **DBScan**.
- Si se quiere entender jerarquías en los datos → **Clustering jerárquico**.
- Si se necesita aprender representaciones complejas → **SOM (Self-Organizing Maps)**.

1.3. Ejemplo en Python – Creación de un Conjunto de Datos Sintético

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

# Generar dataset sintético con 3 clusters
X, y = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=42)

# Graficar los datos sin etiquetas
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.title("Datos sin etiquetar para Clustering")
plt.show()
```

Conclusión: Se observa que los datos tienen patrones naturales que pueden ser agrupados mediante clustering.

Ejercicio para la Sección 1

Objetivo: Comprender cómo se distribuyen los datos antes de aplicar clustering.

Instrucciones:

1. Genera un dataset con 4 grupos utilizando `make_blobs`.
2. Visualiza los datos en un gráfico de dispersión.
3. Reflexiona sobre qué técnica de clustering sería la más adecuada para estos datos.

Pista para la solución en Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

# Generar dataset con 4 clusters
X, y = make_blobs(n_samples=400, centers=4, cluster_std=1.2, random_state=42)

# Visualizar los datos
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.title("Datos Sintéticos para Clustering")
plt.show()
```

Preguntas de Reflexión

1. ¿Cómo podríamos determinar automáticamente el número óptimo de clusters?
2. ¿Qué ventajas tiene visualizar los datos antes de aplicar clustering?
3. ¿Qué pasaría si los clusters estuvieran muy superpuestos?

Sección 2: Algoritmo K-Means

Objetivo: Comprender el funcionamiento del algoritmo K-Means, su proceso iterativo y cómo aplicarlo en un conjunto de datos.

2.1. ¿Qué es K-Means?

- K-Means es un algoritmo de clustering particional que divide los datos en K grupos basados en la minimización de la variación dentro de cada cluster.
- Se basa en la **distancia euclidiana** para asignar puntos a los clusters más cercanos.
- Es uno de los métodos más utilizados debido a su simplicidad y eficiencia computacional.

Aplicaciones de K-Means

- Segmentación de clientes en marketing.
- Agrupamiento de imágenes para clasificación.
- Compresión de datos en bases de datos.
- Análisis de comportamiento de usuarios en aplicaciones web.

2.2. Funcionamiento del Algoritmo

El proceso de K-Means sigue estos pasos:

1. Seleccionar el número de clusters K.
2. Inicializar los centroides de forma aleatoria.
3. Asignar cada punto al centroide más cercano.
4. Actualizar los centroides recalculando la media de cada cluster.
5. Repetir los pasos 3 y 4 hasta que los centroides no cambien significativamente.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# Generar datos sintéticos con 3 clusters
X, y = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=42)

# Aplicar K-Means con 3 clusters
kmeans = KMeans(n_clusters=3, random_state=42)
y_kmeans = kmeans.fit_predict(X)

# Graficar clusters y centroides
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, cmap="viridis", alpha=0.6)
plt.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1],
            c="red", marker="X", s=200, label="Centroides")
plt.title("Clustering con K-Means")
plt.legend()
plt.show()
```

Conclusión:

- Cada color representa un cluster asignado por K-Means.
- Los centroides en rojo indican los centros de cada grupo.

2.3. Elección del Número de Clusters K

Uno de los desafíos de K-Means es elegir el número óptimo de clusters. Los principales métodos son:

1. **Método del Codo (Elbow Method):** Calcula la suma de errores cuadrados para diferentes valores de K.
2. **Índice de Silhouette:** Mide la cohesión y separación de los clusters.

Ejemplo en Python – Método del Codo

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Generar datos
X, y = make_blobs(n_samples=300, centers=5, cluster_std=1.0, random_state=42)

# Calcular la inercia para diferentes valores de K
inercia = []
valores_k = range(1, 10)
for k in valores_k:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    inercia.append(kmeans.inertia_)

# Graficar el método del codo
plt.plot(valores_k, inercia, marker="o", linestyle="-")
plt.xlabel("Número de Clusters (K)")
plt.ylabel("Inercia")
plt.title("Método del Codo para Selección de K")
plt.show()
```

Conclusión: Se elige el punto donde la inercia deja de disminuir abruptamente, es decir, el "codo" de la curva.

Ejercicio para la Sección 2

Objetivo: Aplicar K-Means a un dataset real y determinar el número óptimo de clusters.

Instrucciones:

1. Carga el dataset de pingüinos de Seaborn (`sns.load_dataset("penguins")`).
2. Selecciona variables numéricas (ejemplo: `bill_length_mm`, `bill_depth_mm`).
3. Usa el método del codo para determinar el mejor K.
4. Aplica K-Means y visualiza los clusters.
5. Explica qué insights puedes obtener de la segmentación.

Pista para la solución en Python

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Cargar dataset de pingüinos
df = sns.load_dataset("penguins").dropna()
X = df[["bill_length_mm", "bill_depth_mm"]]

# Método del Codo
inercia = []
valores_k = range(1, 10)
for k in valores_k:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    inercia.append(kmeans.inertia_)

plt.plot(valores_k, inercia, marker="o", linestyle="-")
plt.xlabel("Número de Clusters (K)")
plt.ylabel("Inercia")
plt.title("Método del Codo - Pingüinos")
plt.show()

# Aplicar K-Means con K óptimo
kmeans = KMeans(n_clusters=3, random_state=42)
df["Cluster"] = kmeans.fit_predict(X)

sns.scatterplot(x=df["bill_length_mm"], y=df["bill_depth_mm"],
                hue=df["Cluster"], palette="viridis")
plt.title("Clustering en Pingüinos")
plt.show()
```

Preguntas de Reflexión

1. ¿Cómo afecta la selección de K a la segmentación de datos?
2. ¿Qué limitaciones tiene K-Means cuando los clusters no son esféricos?
3. ¿Cómo podríamos mejorar los resultados de K-Means en datasets complejos?

Sección 3: DBScan y Clustering Basado en Densidad

Objetivo: Comprender el funcionamiento del algoritmo DBScan, sus ventajas sobre K-Means y cómo aplicarlo en Python.

3.1. ¿Qué es DBScan?

DBScan (*Density-Based Spatial Clustering of Applications with Noise*) es un algoritmo de clustering basado en densidad que **no requiere especificar el número de clusters**. Identifica áreas de alta densidad separadas por regiones de baja densidad, lo que permite detectar clusters de formas arbitrarias y manejar outliers de manera más efectiva que K-Means.

Ventajas de DBScan

- No requiere definir K de antemano.
- Puede detectar clusters con formas irregulares.
- Maneja bien el ruido y outliers en los datos.

Desventajas de DBScan

- No funciona bien si las densidades de los clusters son muy diferentes.
- Puede ser lento en datasets grandes.

3.2. Funcionamiento del Algoritmo

El algoritmo DBScan se basa en dos parámetros clave:

- **ϵ (Epsilon):** Determina la distancia máxima entre puntos vecinos.
- **minPts (Mínimo de puntos por cluster):** Define cuántos puntos se requieren para formar un grupo denso.

Cómo funciona:

1. Se selecciona un punto y se busca cuántos vecinos tiene en un radio ϵ .

2. Si hay al menos minPts vecinos, el punto se convierte en "punto núcleo" y se expande el cluster.
3. Si el punto tiene menos vecinos, se marca como "punto de ruido" (outlier).
4. Se repite hasta que todos los puntos han sido visitados.

3.3. Ejemplo en Python – Aplicando DBScan

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons

# Generar datos con forma de media luna (difícil para K-Means)
X, y = make_moons(n_samples=300, noise=0.05, random_state=42)

# Aplicar DBScan
dbscan = DBSCAN(eps=0.2, min_samples=5)
clusters = dbscan.fit_predict(X)

# Visualizar resultados
plt.scatter(X[:, 0], X[:, 1], c=clusters, cmap="viridis", alpha=0.7)
plt.title("Clustering con DBScan")
plt.show()
```

Conclusión:

- DBScan detecta formas de clusters más complejas que K-Means.
- Los puntos marcados como -1 son outliers que no pertenecen a ningún cluster.

3.4. Comparación entre K-Means y DBScan

Característica	K-Means	DBScan
Tipo de Clustering	Basado en particiones	Basado en densidad
Forma de los Clusters	Esféricos	Arbitrarias
Sensibilidad a Outliers	Alta	Baja
Número de Clusters	Debe definirse antes (K)	Se detecta automáticamente
Uso Recomendado	Cuando los clusters son bien definidos	Cuando hay formas complejas y ruido

¿Cuándo usar DBScan?

- Si los clusters tienen formas no esféricas.
- Si hay valores atípicos en los datos.
- Si no sabemos de antemano el número de clusters.

Ejercicio para la Sección 3

Objetivo: Aplicar DBScan a un dataset real y comparar los resultados con K-Means.

Instrucciones:

1. Carga el dataset de flores Iris (`sns.load_dataset("iris")`).
2. Selecciona dos características (`sepal_length` y `sepal_width`).
3. Aplica DBScan y visualiza los clusters.
4. Aplica K-Means y compara los resultados.
5. Explica qué diferencias observas entre ambos métodos.

Pista para la solución en Python

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, DBSCAN

# Cargar dataset Iris
df = sns.load_dataset("iris")
X = df[["sepal_length", "sepal_width"]].values

# Aplicar K-Means
kmeans = KMeans(n_clusters=3, random_state=42)
df["Cluster_KMeans"] = kmeans.fit_predict(X)

# Aplicar DBScan
dbscan = DBSCAN(eps=0.5, min_samples=5)
df["Cluster_DBScan"] = dbscan.fit_predict(X)

# Visualizar
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.scatterplot(x=df["sepal_length"], y=df["sepal_width"],
                hue=df["Cluster_KMeans"], palette="viridis")
plt.title("Clustering con K-Means")

plt.subplot(1, 2, 2)
sns.scatterplot(x=df["sepal_length"], y=df["sepal_width"],
                hue=df["Cluster_DBScan"], palette="viridis")
plt.title("Clustering con DBScan")
plt.show()
```

Preguntas de Reflexión

- 1. ¿Cuántos clusters detectó DBScan en comparación con K-Means?
- 2. ¿DBScan detectó puntos como outliers? ¿Por qué?
- 3. ¿Qué ventajas ofrece DBScan en este caso?

Sección 4: Evaluación de la Calidad del Clustering

Objetivo: Aprender a medir la calidad de los clusters generados con diferentes algoritmos mediante métricas de evaluación.

4.1. ¿Por qué evaluar un Clustering?

A diferencia del aprendizaje supervisado, en clustering no tenemos etiquetas verdaderas, lo que hace difícil evaluar si los clusters son correctos. Existen métricas que miden la **cohesión interna** y la **separación entre clusters**.

Métricas más utilizadas en clustering

Métrica	Descripción	Cuándo Usarla
Inercia (SSE)	Mide la suma de distancias de cada punto a su centroide (menor es mejor).	Evaluar K-Means.
Coefficiente de Silhouette	Mide cuán similares son los puntos dentro de un cluster frente a otros (rango -1 a 1, mayor es mejor).	Comparar diferentes algoritmos de clustering.
Índice de Davies-Bouldin (DBI)	Evalúa la compactación de los clusters y la distancia entre ellos (menor es mejor).	Comparar calidad general del clustering.

4.2. Aplicando Métricas de Evaluación en Python

```

import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import silhouette_score, davies_bouldin_score

# Cargar dataset de iris
df = sns.load_dataset("iris")
X = df[["sepal_length", "sepal_width"]].values

# Aplicar K-Means
kmeans = KMeans(n_clusters=3, random_state=42)
df["Cluster_KMeans"] = kmeans.fit_predict(X)

# Aplicar DBScan
dbscan = DBSCAN(eps=0.5, min_samples=5)
df["Cluster_DBScan"] = dbscan.fit_predict(X)

# Evaluar con Silhouette Score
silhouette_kmeans = silhouette_score(X, df["Cluster_KMeans"])
mask = df["Cluster_DBScan"] != -1
silhouette_dbscan = silhouette_score(X[mask], df["Cluster_DBScan"][mask])

# Evaluar con Davies-Bouldin Index
dbi_kmeans = davies_bouldin_score(X, df["Cluster_KMeans"])
dbi_dbscan = davies_bouldin_score(X[mask], df["Cluster_DBScan"][mask])

print(f"Silhouette - K-Means: {silhouette_kmeans:.2f}")
print(f"Silhouette - DBScan: {silhouette_dbscan:.2f}")
print(f"Davies-Bouldin - K-Means: {dbi_kmeans:.2f}")
print(f"Davies-Bouldin - DBScan: {dbi_dbscan:.2f}")

```

Conclusión:

- Un mayor coeficiente de Silhouette indica clusters bien formados.
- Un menor Davies-Bouldin Index significa clusters más compactos y separados.

4.3. Interpretación de Resultados**¿Qué hacer si la métrica indica un clustering deficiente?**

- Probar otro algoritmo de clustering (si K-Means falla, intentar DBScan).
- Ajustar hiperparámetros (K en K-Means o ϵ en DBScan).
- Aplicar reducción de dimensionalidad (PCA) para mejorar la separación de clusters.

```

import matplotlib.pyplot as plt
from sklearn.metrics import silhouette_score

# Probar K-Means con diferentes valores de K
valores_k = range(2, 7)
silhouette_scores = []
for k in valores_k:
    kmeans = KMeans(n_clusters=k, random_state=42)
    clusters = kmeans.fit_predict(X)
    silhouette_scores.append(silhouette_score(X, clusters))

# Graficar Silhouette Score vs. K
plt.plot(valores_k, silhouette_scores, marker="o")
plt.xlabel("Número de Clusters (K)")
plt.ylabel("Coeficiente de Silhouette")
plt.title("Evaluación de Clustering con Silhouette Score")
plt.show()

```

Conclusión:

- Un pico en la curva indica el número óptimo de clusters.
- Si no hay un claro máximo, puede ser útil probar otros algoritmos.

Ejercicio para la Sección 4

Objetivo: Comparar diferentes algoritmos de clustering con métricas de evaluación.

Instrucciones:

- 1. Carga el dataset de diamantes (`sns.load_dataset("diamonds")`).
- 2. Selecciona variables numéricas (`carat` y `price`).
- 3. Aplica K-Means y DBScan.
- 4. Calcula el Coeficiente de Silhouette y el Davies-Bouldin Index para ambos.
- 5. Explica cuál algoritmo obtuvo la mejor segmentación.

Pista para la solución en Python

```
import seaborn as sns
import numpy as np
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import silhouette_score, davies_bouldin_score

# Cargar dataset de diamantes (muestra)
df = sns.load_dataset("diamonds")
X = df[["carat", "price"]].values[:1000]

# Aplicar K-Means
kmeans = KMeans(n_clusters=3, random_state=42)
labels_km = kmeans.fit_predict(X)

# Aplicar DBScan
dbscan = DBSCAN(eps=5000, min_samples=5)
labels_db = dbscan.fit_predict(X)

# Métricas
mask = labels_db != -1
print(f"Silhouette - K-Means: {silhouette_score(X, labels_km):.2f}")
print(f"Silhouette - DBScan: {silhouette_score(X[mask], labels_db[mask]):.2f}")
print(f"Davies-Bouldin - K-Means: {davies_bouldin_score(X, labels_km):.2f}")
print(f"Davies-Bouldin - DBScan: {davies_bouldin_score(X[mask], labels_db[mask]):.2f}")
```

Preguntas de Reflexión

- 1. ¿Cuál algoritmo obtuvo el mejor coeficiente de Silhouette?
- 2. ¿El Davies-Bouldin Index confirma los resultados del Silhouette Score?
- 3. ¿En qué situaciones puede fallar cada métrica?

Sección 5: Ejercicio Integrador Obligatorio

Entregar en la Tarea asignada en el campus. La misma se evaluará según la rúbrica publicada en la tarea.

Objetivo: Aplicar todas las técnicas de clustering vistas en la clase y compartir los resultados con la comunidad para discutir diferentes enfoques.

5.1. Ejercicio Integrador

Contexto: Supongamos que trabajamos en el departamento de análisis de datos de una empresa de e-commerce y necesitamos identificar grupos de clientes con patrones de compra similares para diseñar estrategias de marketing personalizadas.

Dataset a utilizar: [UCI Online Retail Dataset](#)

Contiene transacciones de una tienda online británica entre 2010 y 2011.

Variables principales:

Variable	Descripción
InvoiceNo	Número de factura
StockCode	Código del producto
Description	Descripción del producto

Variable	Descripción
Quantity	Cantidad comprada
InvoiceDate	Fecha de la factura
UnitPrice	Precio unitario
CustomerID	Identificador del cliente
Country	País del cliente

Tareas a realizar

1. **Preprocesamiento de Datos**
- Eliminar valores nulos y registros con cantidades negativas.
 - Agrupar por **CustomerID** para obtener métricas por cliente: TotalCompras, CantidadTotal, GastoTotal y Frecuencia.
2. **Codificación y Normalización**
- Convertir Frecuencia en una variable ordinal (Alta=2, Media=1, Baja=0, definida por terciles).
 - Normalizar CantidadTotal y GastoTotal usando MinMaxScaler o StandardScaler.
3. **Aplicación de Algoritmos de Clustering**
- Aplicar K-Means y determinar el número óptimo de clusters con el método del codo.
 - Aplicar DBScan ajustando **eps** y **min_samples** para detectar clientes "outliers".
4. **Evaluación del Clustering**
- Comparar resultados con Silhouette Score y Davies-Bouldin Index.
 - Justificar cuál método produce mejor segmentación de clientes y por qué.
5. **Interacción con una Herramienta de IA (LLM)**
- Consulta sugerida: "Tengo un dataset de transacciones de clientes de e-commerce. ¿Qué técnicas de clustering me recomendarías para segmentarlos?"
 - Comparar la respuesta del LLM con lo aprendido en clase y reflexionar sobre su utilidad.

Reflexión Final de la Clase

- ¿Cuál algoritmo de clustering fue más efectivo en este ejercicio?
- ¿Cómo podríamos mejorar la precisión de la segmentación?
- ¿Qué limitaciones tiene el clustering en la minería de datos?

Bibliografía

1. Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques* (3rd ed.). Elsevier.

2. McKinney, W. (2017). *Python for Data Analysis*. O'Reilly Media.

3. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

4. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.

5. Kaufman, L., & Rousseeuw, P. J. (2005). *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.

6. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*. Springer.

7. Provost, F., & Fawcett, T. (2013). *Data Science for Business*. O'Reilly Media.

8. Seaborn Documentation. (n.d.). Recuperado de <https://seaborn.pydata.org/>.

9. Scikit-learn Developers. (n.d.). Recuperado de <https://scikit-learn.org/>.

7. Repaso

Clase N° 7: Repaso de Conceptos

1. Presentación

A lo largo de las sesiones anteriores, hemos recorrido los conceptos fundamentales de la Minería de Datos, desde qué es y para qué sirve hasta las primeras técnicas de preparación y análisis exploratorio de datos, así como la introducción a técnicas de modelado como el clustering y la visualización.

El objetivo de esta clase es consolidar los conocimientos adquiridos, revisar los flujos de trabajo típicos en un proyecto de minería de datos y repasar las herramientas básicas y técnicas iniciales.

En esta clase, repasaremos:

- Los conceptos de Minería de Datos, su relación con otras disciplinas y sus aplicaciones.
- El proceso de descubrimiento de conocimiento (KDD) y el modelo CRISP-DM.
- Las técnicas esenciales de preparación de datos, incluyendo limpieza y transformación.
- El análisis exploratorio de datos (EDA) en sus modalidades univariada y multivariada.
- Las técnicas básicas de visualización de datos y sus herramientas.
- El concepto de Clustering y reducción de dimensionalidad (PCA).

Desarrollo

Conceptos Fundamentales de Minería de Datos

La minería de datos es una disciplina que se enfoca en el descubrimiento de patrones, tendencias y correlaciones dentro de grandes volúmenes de datos. Utiliza técnicas de diversas áreas como la Estadística, el Aprendizaje Automático, Bases de Datos y la Inteligencia Artificial.

Es importante diferenciar entre **Dato**, **Información** y **Conocimiento**:

Concepto	Definición	Ejemplo
Dato	Unidad mínima de información sin contexto.	"25°C"
Información	Dato procesado con significado.	"La temperatura actual es de 25°C"
Conocimiento	Interpretación de la información para la toma de decisiones.	"Dado que hace calor, la venta de bebidas frías aumentará"

Los proyectos de minería de datos suelen seguir metodologías estructuradas:

Proceso KDD (Knowledge Discovery in Databases) — 5 fases:

1. **Selección de datos:** Elegir los datos relevantes.
2. **Preprocesamiento y limpieza:** Manejar errores, duplicados, inconsistencias y valores faltantes.
3. **Transformación de datos:** Convertir datos a un formato adecuado para el análisis.
4. **Minería de datos:** Aplicar algoritmos para encontrar patrones.
5. **Evaluación e interpretación:** Analizar la utilidad de los patrones descubiertos.

Modelo CRISP-DM — 6 fases:

1. **Comprensión del negocio:** Definir objetivos.
2. **Comprensión de los datos:** Analizar fuentes y calidad.
3. **Preparación de los datos:** Limpieza y transformación.
4. **Modelado:** Aplicar algoritmos.
5. **Evaluación:** Validar modelos.
6. **Despliegue:** Implementar modelos para uso real.

Ejercicio 1: Repaso de Fundamentos y Procesos

Supongamos que un servicio de streaming quiere reducir la "fuga" de clientes. Para ello, planean usar minería de datos.

1. **Identifica:**
 - ¿Qué tipo de dato podrían recopilar?
 - ¿Qué información podrían derivar de esos datos?
 - ¿Qué conocimiento útil podrían extraer para reducir la fuga?
2. **Relaciona:** Describe brevemente qué acciones de este escenario corresponden a:
 - Fase de Selección de Datos en KDD.
 - Fase de Modelado en CRISP-DM.
 - Fase de Despliegue en CRISP-DM.

Preparación y Análisis Exploratorio de Datos

Antes de aplicar algoritmos, es crucial preparar los datos. El **limpieza de datos** aborda problemas como valores nulos, duplicados e inconsistencias. Las técnicas para manejar datos faltantes incluyen la eliminación de registros o columnas y la imputación (media, mediana, moda

o interpolación). La detección de **outliers** también es clave, usando métodos como IQR o boxplots.

La **transformación de datos** incluye normalización, estandarización y codificación de variables categóricas (One-Hot Encoding o Label Encoding).

El **Análisis Exploratorio de Datos (EDA)** involucra:

- **Análisis Univariado:** Examina una variable a la vez (histogramas, boxplots, media, mediana, desviación estándar).
- **Análisis Multivariado:** Analiza relaciones entre dos o más variables (scatter plots, correlación de Pearson, PCA, clustering).

Ejercicio 2: Preparación y EDA Básica

Considera el siguiente conjunto de datos de clientes:

Cliente	Edad	Ingresos	Reclamos	Estado_Cuenta
1	25	50000	1	Pagado
2	40	60000	0	Pagado
3	32	70000	2	Moroso
4	-5	80000	3	Moroso
5	30	45000	0	Pagado
6	45	NaN	2	Pagado

1. Identifica los problemas de calidad en este conjunto de datos (valores faltantes, inconsistentes).
2. Sugiere cómo manejarías estos problemas (eliminar, imputar, corregir).
3. Describe qué tipo de análisis exploratorio realizarías para entender la distribución de "Ingresos" y la relación entre "Edad" e "Ingresos".
¿Qué gráficos usarías?

Técnicas Clave de Análisis: Clustering y Reducción de Dimensionalidad

El **Clustering** es una técnica de aprendizaje no supervisado que busca dividir los datos en grupos homogéneos (clusters) basados en su similitud. Tiene aplicaciones amplias como la segmentación de clientes.

Algoritmos de clustering vistos:

Algoritmo	Descripción	Ventajas / Limitaciones
K-Means	Método particional que divide datos en K clusters basados en la distancia a centroides.	Simple y eficiente; requiere definir K y es sensible a la inicialización.
DBScan	Algoritmo basado en densidad; agrupa puntos próximos y detecta ruido.	Ideal para formas irregulares; no necesita K, pero es sensible a parámetros de densidad.
SOM (Self-Organizing Maps)	Redes neuronales para visualización y reducción de dimensionalidad, preservando la topología.	Encuentra patrones ocultos; su interpretación puede ser menos intuitiva.

El **Análisis de Componentes Principales (PCA)** se utiliza para la reducción de dimensionalidad. PCA transforma variables correlacionadas en un conjunto más pequeño de componentes principales no correlacionadas, ayudando a reducir variables, facilitar la visualización y mejorar la eficiencia de los modelos. La normalización previa es importante para PCA.

Ejercicio 3: Conceptos de Clustering y PCA

1. Tienes un conjunto de datos de clientes con muchas variables numéricas (edad, ingresos, frecuencia de compra, etc.). ¿Por qué podrías considerar aplicar PCA antes de un análisis de clustering o modelado predictivo?
2. Quieres segmentar a tus clientes en grupos distintos, pero no sabes cuántos grupos existen naturalmente en tus datos. ¿Qué técnica de clustering de las vistas (K-Means, SOM, DBScan) podría ser más adecuada para este caso y por qué?

Visualización de Datos

La visualización de datos es una herramienta fundamental en minería de datos que permite comprender rápidamente grandes volúmenes de información, detectar patrones, identificar valores atípicos y comunicar hallazgos de manera clara.

Tipos de gráficos y cuándo usarlos:

Gráfico	Uso
Histograma	Distribución de una variable numérica.
Boxplot	Detección de valores atípicos y distribución de una variable.
Scatter Plot	Relación entre dos variables numéricas.
Bar Chart	Comparación de categorías.
Line Plot	Evolución en el tiempo.
Heatmap	Correlación entre múltiples variables (matrices de correlación).
Pairplot	Relaciones entre múltiples variables numéricas en un solo gráfico.
Violin Plot	Distribución de datos dentro de cada categoría (mezcla de boxplot e histograma).

Matplotlib vs Seaborn: Matplotlib es más básica y flexible (control total). Seaborn ofrece gráficos estadísticos más atractivos con menos código.

Ejercicio 4: Aplicación de Visualización

Considera el conjunto de datos de clientes corregido (valor faltante de Ingresos en Cliente 6 imputado a 65000; edad inconsistente del Cliente 4 corregida a 35):

Cliente	Edad	Ingresos	Reclamos	Estado_Cuenta
1	25	50000	1	Pagado
2	40	60000	0	Pagado
3	32	70000	2	Moroso
4	35	80000	3	Moroso
5	30	45000	0	Pagado
6	45	65000	2	Pagado

1. ¿Qué tipo de gráfico podrías usar para visualizar la distribución de la variable "Reclamos"?

2. ¿Qué gráfico es ideal para visualizar la relación lineal entre "Edad" e "Ingresos"?

3. Si quisieras ver la correlación entre "Edad", "Ingresos" y "Reclamos" simultáneamente, ¿qué tipo de visualización de Seaborn sería adecuada?

Ejercicio de Repaso Integrador

Estás a cargo de un proyecto de minería de datos para una empresa de telecomunicaciones que busca predecir la probabilidad de que un cliente abandone el servicio (churn). Describe el proceso general que seguirías, haciendo referencia a los conceptos y metodologías vistos en las clases:

- **Etap**

a de Comprensión (Negocio y Datos):

¿Qué harías?

• **Etap**

a de Preparación de Datos:

¿Qué problemas podrías encontrar y cómo los abordarías?

• **Etap**

a de Análisis Exploratorio:

¿Qué buscarías y qué visualizaciones usarías?

• **Etap**

a de Modelado:

¿Qué tipo de técnica aplicarías y por qué?

• **Etap**

a de Evaluación y Despliegue:

¿Qué implica brevemente?

3. Cierre de la Clase de Repaso

Hemos revisado los pilares de la minería de datos, desde la definición y los procesos estándar hasta las etapas iniciales de preparación, exploración y visualización de datos, y la introducción a técnicas de clustering y PCA. Recordemos la importancia de la calidad de los datos y de visualizar para comprender y comunicar hallazgos. Las metodologías KDD y CRISP-DM ofrecen un marco sólido para abordar los proyectos de manera estructurada.

Bibliografía

1. Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques* (3rd ed.). Elsevier.

2. Hernández Orallo, J., Ramírez Quintana, M. J., & Ferri Ramírez, C. (2004). *Introducción a la Minería de Datos*. Pearson-Prentice Hall.

3. Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J. (2017). *Data Mining: Practical Machine Learning Tools and Techniques*. Morgan Kaufmann.

4. Layton, R. (2017). *Learning Data Mining with Python* (2nd ed.). Packt Publishing.

5. Scikit-learn Developers. (n.d.). Recuperado de <https://scikit-learn.org/>

6. Seaborn Documentation. (n.d.). Recuperado de <https://seaborn.pydata.org/>

7. Matplotlib Developers. (n.d.). Recuperado de <https://matplotlib.org/>

8. Aprendizaje Supervisado, Regresion

Clase N° 8: Introducción al Aprendizaje Supervisado, Regresión Logística y Regresión Múltiple

Introducción

Objetivo de la Clase

El aprendizaje supervisado es una de las técnicas más utilizadas en minería de datos y aprendizaje automático. Se basa en entrenar un modelo utilizando **datos etiquetados**, es decir, datos en los que conocemos la respuesta correcta.

En esta clase, aprenderemos a:

- Entender qué es el aprendizaje supervisado y sus diferencias con el no supervisado.
- Distinguir entre problemas de clasificación y regresión.
- Explorar la regresión logística para problemas de clasificación binaria.
- Aplicar la regresión múltiple para predecir valores continuos.
- Evaluar modelos de regresión y clasificación en Python.

Aplicaciones del Aprendizaje Supervisado

- **Medicina:** Diagnóstico de enfermedades basado en síntomas (Clasificación).
- **Finanzas:** Predicción del valor de una acción en la bolsa (Regresión).
- **Marketing:** Identificación de clientes que responderán a una campaña publicitaria (Clasificación).
- **Industria Automotriz:** Predicción del consumo de combustible de un automóvil (Regresión).

Ejemplo real: Una aseguradora quiere predecir si un cliente presentará un reclamo el próximo año (Sí/No) → Regresión Logística. También puede querer predecir el monto del reclamo → Regresión Múltiple.

Sección 1: Introducción al Aprendizaje Supervisado

Aprendizaje Supervisado vs. No Supervisado

Tipo de Aprendizaje	Descripción	Ejemplo
Supervisado	Se entrena con datos donde se conoce la respuesta correcta.	Clasificación de correos en spam o no spam.
No Supervisado	El modelo encuentra patrones sin conocer etiquetas previas.	Agrupación de clientes según su comportamiento de compra.

Diferencia entre Clasificación y Regresión

Tipo de Problema	Descripción	Ejemplo
Clasificación	Predice una categoría o clase.	Diagnóstico de enfermedades (Positivo/Negativo).
Regresión	Predice un valor numérico continuo.	Predicción del precio de una casa.

Diagrama de Decisión: ¿Clasificación o Regresión?

- Si la variable a predecir es numérica continua → **Usar Regresión**.
- Si la variable a predecir es una categoría → **Usar Clasificación**.

1.1. Implementación en Python – Clasificación vs. Regresión

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression, LinearRegression

# Datos sintéticos
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y_class = np.array([0, 0, 0, 1, 1, 1, 1, 1, 1, 1])
y_reg = np.array([2, 4, 5, 7, 10, 12, 14, 18, 20, 22])

# Ajustar modelos
clf = LogisticRegression().fit(X, y_class)
reg = LinearRegression().fit(X, y_reg)

X_test = np.linspace(0, 12, 100).reshape(-1, 1)
y_pred_class = clf.predict_proba(X_test)[:, 1]
y_pred_reg = reg.predict(X_test)

# Graficar
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.scatter(X, y_class, color="blue", label="Datos")
plt.plot(X_test, y_pred_class, color="red", label="Modelo Logístico")
plt.title("Clasificación: Regresión Logística")
plt.legend()

plt.subplot(1, 2, 2)
plt.scatter(X, y_reg, color="blue", label="Datos")
plt.plot(X_test, y_pred_reg, color="red", label="Regresión Lineal")
plt.title("Regresión: Predicción de un Valor Continuo")
plt.legend()
plt.show()
```

Conclusión:

- La Regresión Logística estima la probabilidad de una categoría (0 o 1).
- La Regresión Lineal estima un valor continuo.

Ejercicio para la Sección 1

Objetivo: Diferenciar cuándo aplicar clasificación o regresión en un dataset real.

Instrucciones:

1. Carga el dataset del Titanic (`sns.load_dataset("titanic")`).
2. Define si la predicción de supervivencia es clasificación o regresión.
3. Aplica un modelo de Regresión Logística para predecir la supervivencia.
4. Aplica una Regresión Lineal para predecir la tarifa del boleto según la edad.
5. Explica qué modelo fue más apropiado en cada caso.

Pista para la solución en Python

```
import seaborn as sns
import pandas as pd
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.model_selection import train_test_split

df = sns.load_dataset("titanic").dropna()

X_class = df[["age", "fare"]]
y_class = df["survived"]
X_reg = df[["age"]]
y_reg = df["fare"]

X_tr_c, X_te_c, y_tr_c, y_te_c = train_test_split(X_class, y_class, test_size=0.2, random_state=42)
X_tr_r, X_te_r, y_tr_r, y_te_r = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)

clf = LogisticRegression(max_iter=1000).fit(X_tr_c, y_tr_c)
reg = LinearRegression().fit(X_tr_r, y_tr_r)

print(f"Precisión Logística: {clf.score(X_te_c, y_te_c):.2f}")
print(f"R² Lineal: {reg.score(X_te_r, y_te_r):.2f}")
```

Preguntas de Reflexión

- 1. ¿Por qué es más adecuado usar Regresión Logística para predecir la supervivencia?
- 2. ¿Por qué la Regresión Lineal es más apropiada para predecir la tarifa del boleto?
- 3. ¿Qué características adicionales podríamos incluir para mejorar los modelos?

Sección 2: Regresión Logística

2.2. Función Sigmoide

La función sigmoide convierte cualquier valor en un rango de 0 a 1, facilitando la interpretación como probabilidad.

- Si el resultado es cercano a **1**, el modelo predice clase 1 (ejemplo: "Aprobado").
- Si es cercano a **0**, predice clase 0 (ejemplo: "Rechazado").
- Se establece un umbral de clasificación, típicamente **0.5**.

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

z = np.linspace(-10, 10, 100)
plt.plot(z, sigmoid(z), label="Sigmoide", color="blue")
plt.axvline(0, color="gray", linestyle="--", label="Umbral (0)")
plt.axhline(0.5, color="red", linestyle="--", label="Probabilidad = 0.5")
plt.xlabel("Z")
plt.ylabel("Probabilidad")
plt.title("Función Sigmoide en Regresión Logística")
plt.legend()
plt.show()
```

2.3. Evaluación del Modelo

Métricas para Regresión Logística

Métrica	Descripción
Matriz de Confusión	Muestra verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos.
Precisión	Proporción de predicciones correctas sobre el total.
Recall (Sensibilidad)	Capacidad del modelo para identificar correctamente la clase positiva.
Curva ROC-AUC	Evalúa el desempeño comparando Tasa de Verdaderos Positivos vs. Falsos Positivos.

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, classification_report

y_real = [1, 0, 1, 1, 0, 1, 0, 0, 1, 0]
y_pred = [1, 0, 1, 0, 0, 1, 0, 1, 1, 0]

cm = confusion_matrix(y_real, y_pred)
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Clase 0", "Clase 1"],
            yticklabels=["Clase 0", "Clase 1"])
plt.xlabel("Predicción")
plt.ylabel("Realidad")
plt.title("Matriz de Confusión")
plt.show()

print("
Reporte de Clasificación:
", classification_report(y_real, y_pred))
```

Conclusión: La matriz de confusión nos ayuda a ver los errores de clasificación. En salud, es mejor minimizar falsos negativos.

Ejercicio para la Sección 2

Objetivo: Aplicar Regresión Logística en un problema real.

Instrucciones:

1. Carga el dataset del Titanic.
2. Selecciona **age**, **fare** y **sex** como variables predictoras.
3. Transforma **sex** en variable numérica (0 = femenino, 1 = masculino).
4. Aplica Regresión Logística para predecir la supervivencia.
5. Evalúa el modelo con una matriz de confusión y precisión.

Pista para la solución en Python

```
import seaborn as sns
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

df = sns.load_dataset("titanic").dropna()
df["sex"] = df["sex"].map({"male": 1, "female": 0})
X = df[["age", "fare", "sex"]]
y = df["survived"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LogisticRegression(max_iter=1000).fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Precisión:", accuracy_score(y_test, y_pred))
print("
Matriz de Confusión:
", confusion_matrix(y_test, y_pred))
print("
Reporte:
", classification_report(y_test, y_pred))
```

Preguntas de Reflexión

1. ¿Cómo afecta la variable "sexo" a la predicción de supervivencia?
2. ¿Por qué es importante balancear precisión y recall?
3. ¿Cómo podríamos mejorar el modelo agregando más variables?

Sección 3: Regresión Múltiple

Objetivo: Comprender el modelo de Regresión Múltiple, cómo interpretar sus coeficientes y aplicarlo en un caso práctico.

3.1. ¿Qué es la Regresión Múltiple?

La Regresión Múltiple es una extensión de la Regresión Lineal donde se utilizan múltiples variables predictoras para estimar un valor numérico:

$$Y = \beta_0 + \beta_1X_1 + \beta_2X_2 + \dots + \beta_nX_n + \epsilon$$

- **Y**: variable dependiente (lo que queremos predecir).
- **X₁, X₂, ..., X_n**: variables independientes.
- **β₀**: intercepto.
- **β₁, ..., β_n**: coeficientes de cada variable predictora.
- **ε**: error del modelo.

3.2. Interpretación de los Coeficientes

Variable	Coeficiente (β)	Interpretación
Superficie (m²)	500	Por cada metro cuadrado adicional, el precio aumenta en \$500.
Número de habitaciones	1500	Por cada habitación adicional, el precio sube \$1500.
Distancia al centro (km)	-2000	Por cada kilómetro más lejos del centro, el precio baja \$2000.

3.3. Multicolinealidad en Regresión Múltiple

Ocurre cuando dos o más variables independientes están fuertemente correlacionadas, afectando la estabilidad del modelo. Se detecta con la **matriz de correlación** o el **Factor de Inflación de Varianza (VIF)** — un VIF > 10 indica alta multicolinealidad.

```
import pandas as pd
from statsmodels.stats.outliers_influence import variance_inflation_factor

data = {
    "Superficie":      [50, 60, 70, 80, 90, 100, 110, 120],
    "Habitaciones":    [2, 3, 3, 4, 4, 5, 5, 6],
    "Distancia_Centro":[5, 4, 3, 3, 2, 1, 1, 0]
}
df = pd.DataFrame(data)

vif_data = pd.DataFrame()
vif_data["Variable"] = df.columns
vif_data["VIF"] = [variance_inflation_factor(df.values, i) for i in range(df.shape[1])]
print(vif_data)
```

3.4. Evaluación del Modelo

Métricas para Regresión Múltiple

Métrica	Interpretación
R² (Coeficiente de Determinación)	Porcentaje de variabilidad explicada por el modelo (mayor es mejor, máximo 1).
MSE (Error Cuadrático Medio)	Error promedio cuadrático (menor es mejor).
MAE (Error Absoluto Medio)	Promedio de los errores absolutos (menor es mejor).

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

y_real = [200000, 220000, 250000, 270000, 300000]
y_pred = [210000, 225000, 240000, 260000, 295000]

print(f"R²: {r2_score(y_real, y_pred):.2f}")
print(f"MSE: {mean_squared_error(y_real, y_pred):.2f}")
print(f"MAE: {mean_absolute_error(y_real, y_pred):.2f}")
```

Ejercicio para la Sección 3

Instrucciones:

1. Carga un dataset de precios de viviendas.
2. Selecciona variables como superficie, habitaciones y baños como predictoras.
3. Aplica Regresión Múltiple para predecir el precio.
4. Evalúa el modelo con R^2 , MSE y MAE.
5. Analiza si hay multicolinealidad con VIF y corrige si es necesario.

Pista para la solución en Python

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

# Reemplazar con tu dataset real
# df = pd.read_csv("house_prices.csv")
# X = df[["sqft_living", "bedrooms", "bathrooms"]]
# y = df["price"]

# X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# model = LinearRegression().fit(X_train, y_train)
# y_pred = model.predict(X_test)
# print(f"R²: {r2_score(y_test, y_pred):.2f}")
# print(f"MSE: {mean_squared_error(y_test, y_pred):.2f}")
# print(f"MAE: {mean_absolute_error(y_test, y_pred):.2f}")
```

Preguntas de Reflexión

1. ¿Qué variable tiene mayor impacto en el precio de la vivienda?
2. ¿Cómo afecta la multicolinealidad a la interpretación del modelo?
3. ¿Cómo podríamos mejorar el modelo?

Sección 4: Ejercicio Integrador y Publicación en el Foro Obligatorio

Objetivo: Aplicar los conceptos de Regresión Logística y Regresión Múltiple en un caso práctico, evaluar los resultados y compartir las conclusiones en el foro.

4.1. Ejercicio Integrador

Contexto: Trabajas en una empresa de seguros y necesitas desarrollar modelos para:

- Predecir si un cliente presentará un reclamo (Sí/No) → **Regresión Logística**.
- Estimar el monto del reclamo en dólares → **Regresión Múltiple**.

Dataset para el ejercicio

Cliente	Edad	Ingresos (USD)	Historial de Accidentes	Monto Reclamo (USD)	Reclamo (Sí/No)
1	25	30000	1	5000	Sí
2	40	55000	0	0	No
3	32	42000	1	7500	Sí
4	50	60000	0	0	No
5	28	37000	1	6500	Sí
6	45	52000	0	0	No

Tareas a realizar

1. **Preprocesamiento:** Convertir "Reclamo" a numérico (0=No, 1=Sí). Dividir en entrenamiento (80%) y prueba (20%).
2. **Regresión Logística:** Variables predictoras: Edad, Ingresos, Historial de Accidentes. Evaluar con Matriz de Confusión y Precisión.
3. **Regresión Múltiple:** Mismas variables predictoras. Evaluar con R^2 , MSE y MAE.
4. **Interacción con IA (LLM):** Consulta sugerida: "Tengo un dataset de aseguradoras. ¿Qué estrategias de modelado recomendarías para

predecir el monto de un reclamo?"

Código Base para la Solución

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report, r2_score,
                             mean_squared_error, mean_absolute_error)

data = {
    "Edad": [25, 40, 32, 50, 28, 45],
    "Ingresos": [30000, 55000, 42000, 60000, 37000, 52000],
    "Historial_Accidentes": [1, 0, 1, 0, 1, 0],
    "Monto_Reclamo": [5000, 0, 7500, 0, 6500, 0],
    "Reclamo": [1, 0, 1, 0, 1, 0]
}
df = pd.DataFrame(data)

X = df[["Edad", "Ingresos", "Historial_Accidentes"]]
y_c = df["Reclamo"]
y_r = df["Monto_Reclamo"]

X_tr_c, X_te_c, y_tr_c, y_te_c = train_test_split(X, y_c, test_size=0.2, random_state=42)
X_tr_r, X_te_r, y_tr_r, y_te_r = train_test_split(X, y_r, test_size=0.2, random_state=42)

# Regresión Logística
log_model = LogisticRegression(max_iter=1000).fit(X_tr_c, y_tr_c)
y_pred_c = log_model.predict(X_te_c)
print("Precisión:", accuracy_score(y_te_c, y_pred_c))
print("Matriz de Confusión:",
      confusion_matrix(y_te_c, y_pred_c))

# Regresión Múltiple
reg_model = LinearRegression().fit(X_tr_r, y_tr_r)
y_pred_r = reg_model.predict(X_te_r)
print(f"R²: {r2_score(y_te_r, y_pred_r):.2f}")
print(f"MSE: {mean_squared_error(y_te_r, y_pred_r):.2f}")
print(f"MAE: {mean_absolute_error(y_te_r, y_pred_r):.2f}")
```

Bibliografía

1. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
2. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*. Springer.
3. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.
4. McKinney, W. (2017). *Python for Data Analysis*. O'Reilly Media.
5. Kuhn, M., & Johnson, K. (2013). *Applied Predictive Modeling*. Springer.
6. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.
7. Scikit-learn Developers. (n.d.). Recuperado de <https://scikit-learn.org/>

9. Árboles de Decision

Clase N° 9: Introducción a los Árboles de Decisión

Introducción

Objetivo de la Clase

Los árboles de decisión son una de las herramientas más utilizadas en minería de datos y aprendizaje supervisado debido a su interpretabilidad y facilidad de implementación. Permiten dividir datos en subconjuntos con base en condiciones sucesivas, lo que facilita la toma de decisiones automatizadas.

En esta clase aprenderemos a:

- Comprender la estructura y funcionamiento de los árboles de decisión.
- Distinguir entre clasificación y regresión con árboles de decisión.
- Explorar los principales algoritmos de construcción de árboles (ID3, C4.5, CART).
- Implementar un árbol de decisión en Python utilizando scikit-learn.
- Evaluar el rendimiento de un árbol de decisión con métricas adecuadas.

¿Por qué usar Árboles de Decisión?

Los árboles de decisión ofrecen varias ventajas, entre ellas:

- **Fácil interpretación:** Son representables visualmente, lo que facilita su comprensión.
- **Capacidad de manejar datos categóricos y numéricos.**
- **No requieren normalización de datos.**
- **Pueden manejar valores faltantes** en algunos algoritmos como C4.5.

Aplicaciones de los Árboles de Decisión

- **Medicina:** Diagnóstico de enfermedades basado en síntomas.
- **Finanzas:** Predicción de clientes con alto riesgo de impago en préstamos.
- **Marketing:** Segmentación de clientes según su comportamiento de compra.
- **Derecho:** Clasificación de casos jurídicos en categorías específicas.
- **Industria:** Predicción de fallas en maquinarias con base en mediciones sensoriales.

Ejemplo Real

Un banco quiere determinar si un cliente debe recibir un préstamo. Se pueden usar criterios como:

- Ingreso anual (¿superior a \$50,000?)
- Historial de pagos (¿ha tenido deudas impagas?)
- Edad del cliente (¿es menor de 30 años?)

El modelo toma decisiones basadas en estos atributos y asigna una clasificación: "**Aprobado**" o "**Rechazado**".

Sección 1: ¿Qué es un Árbol de Decisión?

Definición

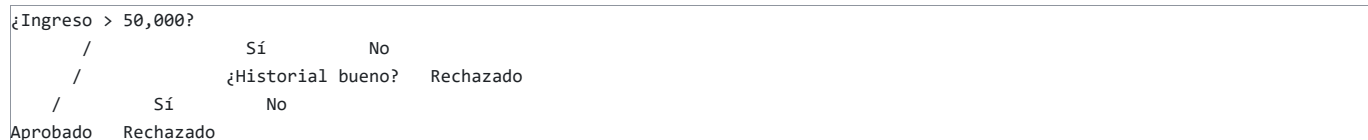
Un árbol de decisión es una técnica de aprendizaje supervisado utilizada para clasificación y regresión, donde los datos se dividen en ramas con base en preguntas sobre sus características.

Estructura de un Árbol de Decisión

Un árbol de decisión está compuesto por:

1. **Nodo Raíz:** Representa el atributo principal sobre el cual se toma la primera decisión.
2. **Nodos Intermedios (de decisión):** Representan criterios de partición sobre diferentes atributos.
3. **Ramas:** Son las conexiones entre los nodos, representando una respuesta a la condición planteada en el nodo.
4. **Nodos Hoja:** Representan la predicción final o resultado del modelo.

Ejemplo: Un Árbol de Decisión para Aprobar un Préstamo



- Si los ingresos son mayores a \$50,000, se considera el historial crediticio.
- Si el historial es bueno, el préstamo es aprobado.
- Si cualquiera de estos factores es negativo, el préstamo es rechazado.

Ejemplo de Código en Python – Visualización de un Árbol de Decisión

```
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree

# Datos ficticios de ingresos y historial crediticio
X = [[60000, 1], [35000, 0], [80000, 1], [20000, 0], [50000, 1]]
y = ["Aprobado", "Rechazado", "Aprobado", "Rechazado", "Aprobado"]

# Crear y entrenar el árbol
tree = DecisionTreeClassifier(criterion="gini", max_depth=3)
tree.fit(X, y)

# Graficar el árbol
plt.figure(figsize=(8, 5))
plot_tree(tree, feature_names=["Ingreso", "Historial"],
          class_names=["Rechazado", "Aprobado"], filled=True)
plt.show()
```

Conclusión:

- El árbol de decisión divide los datos en ramas según condiciones lógicas.
- La representación gráfica ayuda a comprender fácilmente el proceso de toma de decisiones.

1.1. Tipos de Árboles de Decisión

1. **Árboles de Clasificación:** Se utilizan cuando la variable objetivo es categórica (Ejemplo: "Sí" o "No"). Utilizan métricas como Índice Gini o Entropía para determinar las divisiones óptimas. *Ejemplo:* Diagnosticar si un paciente tiene una enfermedad (Sí/No).
2. **Árboles de Regresión:** Se utilizan cuando la variable objetivo es numérica continua. Utilizan métricas como Error Cuadrático Medio (MSE) para minimizar errores en predicciones. *Ejemplo:* Predecir el precio de una casa en función de su tamaño y ubicación.

Comparación entre Clasificación y Regresión

Tipo de Árbol	Variable Objetivo	Ejemplo
Clasificación	Categórica (Sí/No)	Diagnóstico de una enfermedad
Regresión	Continua (Números)	Predicción del precio de una casa

Ejemplo de Código en Python – Comparación de Clasificación vs. Regresión

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor

# Datos simulados
X_class = np.array([[1], [2], [3], [4], [5]])
y_class = np.array(["Bajo", "Bajo", "Medio", "Alto", "Alto"])

X_reg = np.array([[1], [2], [3], [4], [5]])
y_reg = np.array([10000, 15000, 25000, 50000, 70000])

# Modelos
clf = DecisionTreeClassifier(max_depth=2).fit(X_class, y_class)
reg = DecisionTreeRegressor(max_depth=2).fit(X_reg, y_reg)

# Predicción de nuevos valores
print("Clasificación:", clf.predict([[3.5]])) # Esperado: "Medio" o "Alto"
print("Regresión:", reg.predict([[3.5]]))     # Esperado: Un valor numérico intermedio
```

Conclusión:

- Los árboles de clasificación predicen categorías discretas (Ejemplo: "Bajo", "Medio", "Alto").
- Los árboles de regresión predicen valores continuos (Ejemplo: un precio en dólares).

Ejercicio para la Sección 1

Objetivo: Comprender la diferencia entre árboles de clasificación y regresión.

Instrucciones:

1. Cargar el dataset de precios de viviendas (`sns.load_dataset("house_prices")`).
2. Entrenar un árbol de clasificación para predecir si una casa es "barata" o "cara".
3. Entrenar un árbol de regresión para predecir el precio exacto de la casa.
4. Comparar ambos enfoques y explicar sus diferencias.

Preguntas de Reflexión:

1. ¿Cuándo es más útil un árbol de clasificación en lugar de uno de regresión?
2. ¿Cómo afecta la profundidad del árbol a la precisión del modelo?
3. ¿Cómo podríamos mejorar la predicción con más atributos?

Sección 2: Construcción y Algoritmos

2.1. ¿Cómo se construye un Árbol de Decisión?

Un árbol de decisión se construye dividiendo repetidamente los datos en subconjuntos con base en preguntas sobre sus atributos.

Ejemplo: Si queremos clasificar a personas según si compran un automóvil, podríamos hacer preguntas como:

- ¿Tiene ingresos superiores a \$30,000?
- ¿Vive en una ciudad o en zona rural?
- ¿Es mayor de 25 años?

Cada pregunta divide los datos en dos subconjuntos, y este proceso continúa hasta que se llega a un nodo hoja.

Criterios para dividir los datos

Para decidir qué atributo usar en cada nodo, los algoritmos de árboles de decisión utilizan diferentes métricas:

Métrica	Uso	Descripción
Entropía	ID3 y C4.5	Mide la incertidumbre en los datos antes y después de una división.
Ganancia de Información	ID3 y C4.5	Calcula cuánto reduce la entropía una partición.
Índice Gini	CART	Mide la pureza de los nodos después de una partición.
Error Cuadrático Medio (MSE)	Árboles de Regresión	Minimiza la diferencia entre los valores reales y predichos.

Ejemplo: Cálculo de Entropía y Gini

Si tenemos 10 ejemplos y 7 pertenecen a la clase "Sí" y 3 a la clase "No", podemos calcular:

- **Entropía:** $H = -0.7 \cdot \log_2(0.7) - 0.3 \cdot \log_2(0.3) \approx 0.88$ — Una entropía de 0.88 indica que el conjunto es relativamente mixto (incierto).
- **Índice Gini:** $Gini = 1 - (0.7^2 + 0.3^2) = 1 - (0.49 + 0.09) \approx 0.42$ — Un índice Gini de 0.42 indica que el conjunto tiene diversidad de clases.

En general, cuanto más baja es la Entropía o el Gini, mejor es la partición.

2.2. Algoritmos Populares para Construcción de Árboles

1. **ID3 (Iterative Dichotomiser 3):** Utiliza Ganancia de Información basada en Entropía para seleccionar los mejores atributos. *Limitaciones:* No maneja atributos numéricos ni valores faltantes. *Ejemplo de Aplicación:* Clasificación de correos en spam y no spam.
2. **C4.5:** Es una mejora de ID3 que puede manejar atributos numéricos y datos faltantes. Utiliza Índice de Ganancia, que ajusta la ganancia de información para evitar sesgos hacia atributos con muchos valores. *Ejemplo de Aplicación:* Diagnóstico médico basado en síntomas.
3. **CART (Classification and Regression Trees):** Utiliza Índice Gini en clasificación y MSE en regresión. Solo genera árboles binarios (cada nodo tiene exactamente dos ramas). *Ejemplo de Aplicación:* Predicción de precios de viviendas.

Comparación de Algoritmos

Algoritmo	Criterio de División	Soporta Atributos Numéricos	Ramas Múltiples
ID3	Ganancia de Información (Entropía)	No	Sí
C4.5	Índice de Ganancia	Sí	Sí

Algoritmo	Criterio de División	Soporta Atributos Numéricos	Ramas Múltiples
CART	Gini / MSE	Sí	No (Binario)

2.3. Implementación en Python: Comparación de Algoritmos

Ejemplo: Comparar ID3 y CART en Clasificación

```
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Cargar dataset Titanic
df = sns.load_dataset("titanic").dropna()

# Preprocesamiento
df["sex"] = df["sex"].map({"male": 1, "female": 0})
X = df[["age", "fare", "sex", "class"]]
y = df["survived"]

# Convertir "class" en variable numérica
X = pd.get_dummies(X, drop_first=True)

# Dividir en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Árbol ID3 (simulado con entropy en scikit-learn)
tree_id3 = DecisionTreeClassifier(criterion="entropy", max_depth=3, random_state=42)
tree_id3.fit(X_train, y_train)
y_pred_id3 = tree_id3.predict(X_test)

# Árbol CART (scikit-learn usa Gini por defecto)
tree_cart = DecisionTreeClassifier(criterion="gini", max_depth=3, random_state=42)
tree_cart.fit(X_train, y_train)
y_pred_cart = tree_cart.predict(X_test)

# Evaluación de precisión
print("Precisión ID3 (Entropía):", accuracy_score(y_test, y_pred_id3))
print("Precisión CART (Gini):", accuracy_score(y_test, y_pred_cart))
```

Conclusión:

- ID3 usa Entropía y suele generar árboles más grandes y detallados.
- CART usa Gini y tiende a ser más eficiente en árboles binarios.

Ejercicio para la Sección 2

Objetivo: Comparar ID3 y CART en un dataset real.

Instrucciones:

1. Cargar el dataset `sns.load_dataset("penguins")`.
2. Aplicar un árbol ID3 (`criterion="entropy"`) para predecir la especie del pingüino.
3. Aplicar un árbol CART (`criterion="gini"`) con los mismos datos.
4. Comparar las precisiones de ambos modelos.

Pista para la solución en Python:

```
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Cargar dataset de pingüinos
df = sns.load_dataset("penguins").dropna()

# Preprocesamiento
X = df[["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]]
y = df["species"]

# Dividir en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Entrenar modelos
tree_id3 = DecisionTreeClassifier(criterion="entropy", max_depth=4, random_state=42)
tree_id3.fit(X_train, y_train)
tree_cart = DecisionTreeClassifier(criterion="gini", max_depth=4, random_state=42)
tree_cart.fit(X_train, y_train)

# Evaluar modelos
print("Precisión ID3:", accuracy_score(y_test, tree_id3.predict(X_test)))
print("Precisión CART:", accuracy_score(y_test, tree_cart.predict(X_test)))
```

Preguntas de Reflexión:

1. ¿Qué algoritmo fue más preciso en este dataset?
2. ¿Cómo influye la profundidad máxima del árbol en la precisión?
3. ¿Cómo podríamos mejorar estos modelos?

Sección 3: Evaluación y Optimización de Árboles de Decisión

Objetivo: Aprender a evaluar la calidad de un árbol de decisión y explorar técnicas para optimizar su desempeño.

3.1. ¿Cómo Evaluar un Árbol de Decisión?

Para Árboles de Clasificación:

- **Precisión (Accuracy):** Proporción de predicciones correctas.
- **Matriz de Confusión:** Muestra los verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos.
- **Precision, Recall y F1-score:** Métricas clave para evaluar la calidad de la clasificación.
- **Curva ROC-AUC:** Indica la capacidad del modelo para separar las clases.

Para Árboles de Regresión:

- **Coefficiente de Determinación (R^2):** Indica qué porcentaje de la variabilidad en los datos es explicada por el modelo.
- **Error Cuadrático Medio (MSE):** Mide el promedio de los errores al cuadrado.
- **Error Absoluto Medio (MAE):** Mide el error medio en términos absolutos.

Actividad: Evaluación de un Árbol de Clasificación en Python

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Evaluación del modelo
y_pred = tree_cart.predict(X_test)
print("Precisión del Árbol de Decisión:", accuracy_score(y_test, y_pred))
print("\nMatriz de Confusión:\n", confusion_matrix(y_test, y_pred))
print("\nReporte de Clasificación:\n", classification_report(y_test, y_pred))

# Visualización de la matriz de confusión
plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt="d", cmap="Blues",
            xticklabels=y.unique(), yticklabels=y.unique())
plt.xlabel("Predicción")
plt.ylabel("Realidad")
plt.title("Matriz de Confusión")
plt.show()
```

Conclusión:

- La matriz de confusión ayuda a visualizar los errores del modelo.
- Precisión y F1-score son útiles para evaluar el desempeño en clasificación.

3.2. Problemas Comunes en Árboles de Decisión

1. **Sobreajuste (Overfitting):** Ocurre cuando el árbol es demasiado profundo y aprende patrones específicos del dataset de entrenamiento, reduciendo su capacidad de generalizar. *Solución:* Limitar la profundidad máxima (`max_depth`), el número mínimo de observaciones en un nodo (`min_samples_split`) y usar poda.
2. **Subajuste (Underfitting):** Ocurre cuando el árbol es demasiado simple y no capta bien las relaciones en los datos. *Solución:* Aumentar la profundidad del árbol o incluir más variables predictoras.
3. **Selección de Atributos No Relevantes:** Si el árbol usa atributos irrelevantes, puede generar predicciones inconsistentes. *Solución:* Aplicar selección de características para elegir solo las más importantes.
4. **Sensibilidad a los Datos Desequilibrados:** Si hay clases desbalanceadas, el árbol puede favorecer la clase mayoritaria. *Solución:* Aplicar técnicas como balanceo de datos o ponderación de clases.

3.3. Optimización de Árboles de Decisión

1. Ajuste de Parámetros (Hyperparameter Tuning)

Se pueden optimizar los parámetros del árbol para evitar sobreajuste o subajuste:

Parámetro	Descripción	Cómo Ajustarlo
<code>max_depth</code>	Profundidad máxima del árbol	Reducir si hay sobreajuste
<code>min_samples_split</code>	Mínimo de muestras necesarias para dividir un nodo	Aumentar para evitar ramas pequeñas
<code>min_samples_leaf</code>	Mínimo de muestras en una hoja	Evita hojas con pocos datos
<code>max_features</code>	Número de atributos considerados en cada división	Reduce la complejidad del árbol

Ejemplo: Ajuste Manual de Parámetros

```
tree_optimized = DecisionTreeClassifier(max_depth=5, min_samples_split=10,
                                       min_samples_leaf=5, random_state=42)
tree_optimized.fit(X_train, y_train)
y_pred_opt = tree_optimized.predict(X_test)
print("Precisión optimizada:", accuracy_score(y_test, y_pred_opt))
```

2. Búsqueda de Parámetros con GridSearchCV

Se puede automatizar la búsqueda de los mejores parámetros con GridSearchCV.

```
from sklearn.model_selection import GridSearchCV

# Definir parámetros
param_grid = {
    "max_depth": [3, 5, 10],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 5, 10]
}

# Aplicar GridSearchCV
grid_search = GridSearchCV(DecisionTreeClassifier(), param_grid, cv=5, scoring="accuracy")
grid_search.fit(X_train, y_train)
print("Mejores parámetros:", grid_search.best_params_)
```

3. Poda de Árboles

- **Poda previa (Pre-pruning):** Limita la profundidad del árbol durante su construcción (`max_depth`).
- **Poda posterior (Post-pruning):** Permite que el árbol crezca completamente y luego elimina ramas innecesarias.

Actividad extensiva: Poda de un Árbol

```
from sklearn.tree import export_text

# Mostrar el árbol antes de poda
print(export_text(tree_cart, feature_names=X.columns.tolist()))

# Aplicar poda reduciendo max_depth
tree_pruned = DecisionTreeClassifier(max_depth=3, random_state=42)
tree_pruned.fit(X_train, y_train)

# Mostrar el árbol después de la poda
print(export_text(tree_pruned, feature_names=X.columns.tolist()))
```

Conclusión:

- Ajustar `max_depth`, `min_samples_split` y `min_samples_leaf` mejora la generalización del modelo.
- GridSearchCV permite encontrar la mejor configuración automáticamente.
- La poda de árboles evita que el modelo aprenda ruido en los datos.

Actividad para la Sección 3

Objetivo: Optimizar un Árbol de Decisión para mejorar su desempeño.

Instrucciones:

1. Cargar el dataset de "diabetes" (`sns.load_dataset("diabetes")`).
2. Entrenar un árbol de decisión estándar y medir su precisión.
3. Optimizar `max_depth` y `min_samples_split` usando GridSearchCV.
4. Comparar el desempeño del modelo original vs. el optimizado.

Pista para la solución en Python:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

# Cargar dataset
df = sns.load_dataset("diabetes").dropna()

# Seleccionar variables
X = df.drop(columns=["outcome"])
y = df["outcome"]

# Dividir en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Entrenar árbol básico
tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)
print("Precisión Árbol Básico:", accuracy_score(y_test, tree.predict(X_test)))

# Optimizar con GridSearchCV
grid_search = GridSearchCV(tree, param_grid, cv=5, scoring="accuracy")
grid_search.fit(X_train, y_train)

# Evaluar modelo optimizado
best_tree = grid_search.best_estimator_
print("Precisión Árbol Optimizado:", accuracy_score(y_test, best_tree.predict(X_test)))
```

Preguntas de Reflexión:

- 1. ¿Cómo cambió la precisión después de la optimización?
- 2. ¿Qué parámetros fueron más importantes en la mejora del modelo?
- 3. ¿Cómo afecta la poda al rendimiento del árbol?

Sección 4: Ejercicio Integrador

Objetivo: Aplicar los conceptos aprendidos sobre Árboles de Decisión en un caso práctico, optimizar su rendimiento y compartir los resultados en el foro.

Ejercicio Integrador: Predicción de Riesgo Crediticio

Contexto: Un banco quiere construir un modelo para predecir si un cliente incumplirá en el pago de un crédito. Los datos incluyen información sobre ingresos, edad, historial crediticio y monto del préstamo solicitado.

Dataset para el ejercicio

Variable	Descripción	Tipo
Edad	Edad del cliente	Númérica
Ingresos (\$USD)	Ingresos anuales en dólares	Númérica
Historial	Cantidad de deudas previas	Númérica
Monto Crédito	Monto del préstamo solicitado	Númérica
Incumplimiento	1 = No pagó el préstamo, 0 = Sí pagó	Categórica (Sí/No)

Tareas a Realizar:

- 1. **Preprocesamiento de Datos:** Normalizar las variables numéricas. Dividir en entrenamiento (80%) y prueba (20%).
- 2. **Entrenar un Árbol de Decisión Básico:** Utilizar `DecisionTreeClassifier` con hiperparámetros por defecto. Evaluar el modelo con Precisión y Matriz de Confusión.
- 3. **Optimizar el Modelo:** Ajustar `max_depth`, `min_samples_split` y `min_samples_leaf` usando GridSearchCV. Evaluar la mejora en precisión y sobreajuste.
- 4. **Comparar con otro Modelo (Regresión Logística o Random Forest):** Entrenar un Random Forest y comparar sus métricas con el árbol de decisión.
- 5. **Interacción con una Herramienta de IA (ChatGPT, Gemini, Copilot):** Consultar cómo mejorar el modelo y comparar la respuesta con lo

aprendido. *Ejemplo de consulta:* "Estoy entrenando un árbol de decisión para predecir incumplimiento de crédito. ¿Qué técnicas de optimización recomendarías?"

4.2. Implementación en Python

Código Base para la Solución

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Simulación de datos de clientes
data = {
    "Edad": np.random.randint(20, 65, 500),
    "Ingresos": np.random.randint(20000, 120000, 500),
    "Historial": np.random.randint(0, 5, 500),
    "Monto_Credito": np.random.randint(5000, 50000, 500),
    "Incumplimiento": np.random.choice([0, 1], size=500, p=[0.7, 0.3]) # 30% de incumplimiento
}
df = pd.DataFrame(data)

# Dividir en entrenamiento y prueba
X = df.drop(columns=["Incumplimiento"])
y = df["Incumplimiento"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Entrenar Árbol de Decisión Básico
tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)

# Predicciones y evaluación
y_pred = tree.predict(X_test)
print("Precisión Árbol de Decisión:", accuracy_score(y_test, y_pred))
print("\nMatriz de Confusión:\n", confusion_matrix(y_test, y_pred))
print("\nReporte de Clasificación:\n", classification_report(y_test, y_pred))

# Optimización con GridSearchCV
param_grid = {
    "max_depth": [3, 5, 10],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 5, 10]
}
grid_search = GridSearchCV(DecisionTreeClassifier(random_state=42), param_grid, cv=5, scoring="accuracy")
grid_search.fit(X_train, y_train)

# Evaluar Árbol Optimizado
best_tree = grid_search.best_estimator_
y_pred_opt = best_tree.predict(X_test)
print("\nPrecisión Árbol Optimizado:", accuracy_score(y_test, y_pred_opt))

# Comparación con Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
print("\nPrecisión Random Forest:", accuracy_score(y_test, y_pred_rf))
```

Conclusión:

- El Árbol de Decisión optimizado debe superar al modelo básico.
- El Random Forest podría mejorar la predicción, pero con mayor complejidad computacional.

ACTIVIDAD OBLIGATORIA: VER TAREA PARA EL ENUNCIADO DE LA ACTIVIDAD.

Bibliografía

1. Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. CRC Press.
2. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer.
3. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning: With Applications in R*. Springer.
4. Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques*. Elsevier.
5. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
6. Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and Statistical Modeling with Python. *SciPy Conference*.
7. Scikit-learn Developers. (n.d.). *Scikit-learn: Machine Learning in Python*. Recuperado de <https://scikit-learn.org/>
8. Seaborn Documentation. (n.d.). *Seaborn: Statistical Data Visualization*. Recuperado de <https://seaborn.pydata.org/>

10. Minería de Texto - TensorFlow

Clase N° 10: Revisión de Temas Anteriores – Minería de Texto y Web con TensorFlow y Scikit-learn

Introducción

La Minería de Texto es una de las áreas más importantes dentro de la Minería de Datos, ya que permite analizar, procesar y extraer información valiosa a partir de datos no estructurados en formato textual. A diferencia de los datos numéricos o estructurados, el texto presenta desafíos como la ambigüedad del lenguaje, la variabilidad gramatical y la necesidad de preprocesamiento.

En esta clase abordaremos los conceptos clave y las herramientas necesarias para aplicar técnicas de Procesamiento de Lenguaje Natural (NLP) con TensorFlow y Scikit-learn, con el objetivo de convertir texto en información estructurada para su análisis y modelado predictivo.

Objetivos de la Clase

- Comprender la importancia y los fundamentos de la Minería de Texto.
- Aplicar técnicas de Procesamiento de Lenguaje Natural (NLP) para limpiar y analizar datos textuales.
- Utilizar modelos de Machine Learning para clasificación de texto.
- Explorar herramientas como Scikit-learn y TensorFlow en la construcción de modelos de minería de texto.

Importancia de la Minería de Texto en la actualidad

Las aplicaciones de la minería de texto han crecido exponencialmente con el auge de los datos digitales. Algunas de sus aplicaciones más comunes incluyen:

- **Análisis de Sentimientos:** Evaluar la percepción pública sobre productos, servicios o temas específicos en redes sociales.
- **Filtrado de Spam:** Detectar correos electrónicos no deseados mediante modelos de clasificación de texto.
- **Medicina:** Extraer información clave de registros médicos y diagnósticos automáticos.
- **Motores de Búsqueda:** Mejorar los resultados mediante análisis semántico y relevancia contextual.
- **Asistentes Virtuales:** Procesar texto y voz para interactuar con los usuarios (ej. Alexa, Google Assistant).

Ejemplo de Aplicación Real

Las empresas de comercio electrónico, como Amazon y MercadoLibre, utilizan Minería de Texto para analizar reseñas de productos y mejorar la experiencia del usuario. Mediante técnicas de análisis de sentimientos, pueden identificar productos con alta satisfacción o detectar problemas recurrentes para mejorar sus servicios.

Sección 1: Minería de Texto – Conceptos Fundamentales

¿Qué es la Minería de Texto?

La Minería de Texto es el proceso de descubrir patrones, tendencias o conocimientos a partir de datos textuales. Se basa en técnicas de Procesamiento de Lenguaje Natural (NLP) y Machine Learning para estructurar, analizar e interpretar grandes volúmenes de texto.

Flujo de Trabajo en Minería de Texto

1. **Preprocesamiento del Texto:**
 - Conversión a minúsculas.
 - Eliminación de signos de puntuación.
 - Tokenización (separación en palabras o frases).

- Eliminación de palabras vacías (stopwords).
 - Lematización o Stemming (reducción de palabras a su raíz).
2. **Extracción de Características:**
- Representación en Bolsa de Palabras (Bag of Words).
 - TF-IDF (Term Frequency - Inverse Document Frequency).
 - Embeddings de Palabras (Word2Vec, FastText, BERT).
3. **Análisis y Modelado:**
- Clasificación de Texto (Ejemplo: Spam vs. No Spam).
 - Análisis de Sentimientos (Ejemplo: Opinión positiva vs. negativa).
 - Agrupamiento de Documentos (Ejemplo: Clustering de noticias similares).

Actividad Guiada de cada paso en Python

```
import nltk
import string
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

# Descargar recursos necesarios
nltk.download("punkt")
nltk.download("stopwords")

# Texto de ejemplo
texto = "¡La minería de texto es fascinante! Permite analizar grandes volúmenes de datos."

# Conversión a minúsculas y eliminación de puntuación
texto = texto.lower().translate(str.maketrans("", "", string.punctuation))

# Tokenización
tokens = word_tokenize(texto)

# Eliminación de palabras vacías
tokens_limpios = [word for word in tokens if word not in stopwords.words("spanish")]
print("Texto Procesado:", tokens_limpios)
```

Explicación del Código:

1. Se convierte el texto a minúsculas para evitar inconsistencias.
2. Se eliminan signos de puntuación, ya que no aportan significado semántico.
3. Se realiza la tokenización, dividiendo el texto en palabras individuales.
4. Se eliminan las palabras vacías (stopwords), como "de", "es", "la", etc.

Ejemplo de Salida:

```
Texto Procesado: ['minería', 'texto', 'fascinante', 'permite', 'analizar', 'grandes', 'volumenes', 'datos']
```

Conclusión: Después del preprocesamiento, el texto está limpio y listo para ser utilizado en modelos de Machine Learning.

1.1. Representaciones de Texto en Machine Learning

Para utilizar texto en modelos de Machine Learning, es necesario convertirlo en vectores numéricos.

1. Bolsa de Palabras (Bag of Words – BoW)

- Se crea una matriz de términos donde cada palabra representa una columna.
- Se cuenta la frecuencia de aparición de cada palabra en el documento.
- No considera el significado ni el orden de las palabras.

Ejemplo de Matriz de BoW

Documento	minería	texto	datos	permite
Doc 1	1	1	1	1
Doc 2	0	1	2	0

2. TF-IDF (Frecuencia de Término – Frecuencia Inversa de Documento)

- Penaliza palabras frecuentes que aparecen en muchos documentos.
- Da más peso a términos que son específicos de un documento en particular.

Actividad de Ejemplo en Python: Vectorización con TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Documentos de ejemplo
documentos = ["La minería de datos es importante",
              "El análisis de datos es clave en la IA"]

# Aplicar TF-IDF
vectorizador = TfidfVectorizer()
tfidf_matriz = vectorizador.fit_transform(documentos)

# Mostrar la matriz
print("Términos:", vectorizador.get_feature_names_out())
print("Matriz TF-IDF:\n", tfidf_matriz.toarray())
```

Ejemplo de Salida:

```
Términos: ['análisis', 'clave', 'datos', 'importante', 'ia', 'minería']
Matriz TF-IDF:
[[0.          0.          0.534522  0.534522  0.          0.534522]
 [0.534522  0.534522  0.267261  0.          0.534522  0.          ]]
```

Conclusión: TF-IDF mejora la representación de texto, dando más importancia a términos relevantes y menos peso a palabras comunes.

Sección 2: Introducción a TensorFlow y Scikit-learn para Minería de Texto

Objetivo de la Sección: En esta sección, exploraremos las principales herramientas para aplicar modelos de Machine Learning en Minería de Texto. En particular, veremos cómo utilizar TensorFlow y Scikit-learn, dos de las bibliotecas más utilizadas para el desarrollo de modelos de aprendizaje automático en Python.

2.1. TensorFlow: Deep Learning para Minería de Texto

¿Qué es TensorFlow?

TensorFlow es una biblioteca de código abierto desarrollada por Google para la implementación de modelos de Deep Learning. Se usa ampliamente en tareas de Procesamiento de Lenguaje Natural (NLP), incluyendo:

- Clasificación de texto (Ejemplo: detección de spam).
- Modelado de lenguaje (Ejemplo: autocompletar oraciones).
- Análisis de sentimientos (Ejemplo: opiniones positivas o negativas).
- Generación de texto (Ejemplo: traducción automática).

Arquitectura de TensorFlow para NLP

El flujo de trabajo típico en TensorFlow para minería de texto incluye:

1. **Preprocesamiento del texto:** Tokenización, eliminación de stopwords y vectorización.
2. **Creación del modelo:** Uso de redes neuronales recurrentes (RNN), redes convolucionales (CNN) o transformers.
3. **Entrenamiento:** Optimización con algoritmos como Adam o RMSprop.
4. **Evaluación y despliegue:** Prueba del modelo con nuevos datos.

Ejemplo en TensorFlow: Creación de una Red Neuronal para Clasificación de Texto

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Embedding, Dense, LSTM
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np

# Datos de entrenamiento
textos = ["Me encanta este producto", "Es un desastre, muy malo",
          "Excelente calidad y buen precio", "No me gustó, fue horrible"]
etiquetas = np.array([1, 0, 1, 0]) # 1 = Positivo, 0 = Negativo

# Tokenización del texto
tokenizer = Tokenizer(num_words=1000, oov_token="<OOV>")
tokenizer.fit_on_texts(textos)
sequences = tokenizer.texts_to_sequences(textos)

# Padding para que todas las secuencias tengan el mismo tamaño
padded_sequences = pad_sequences(sequences, maxlen=5, padding='post')

# Definir el modelo de red neuronal
modelo = keras.Sequential([
    Embedding(input_dim=1000, output_dim=16, input_length=5),
    LSTM(16),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid')
])

# Compilar y entrenar el modelo
modelo.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
modelo.fit(padded_sequences, etiquetas, epochs=5, verbose=1)
```

Explicación del Código:

1. Se tokenizan los textos, convirtiéndolos en secuencias numéricas.
2. Se aplica Padding, asegurando que todas las oraciones tengan la misma longitud.
3. Se define el modelo con una capa Embedding, seguida de una capa LSTM (Long Short-Term Memory).
4. Se entrena la red neuronal para clasificación binaria (positivo/negativo).

Conclusión:

- Las redes neuronales permiten capturar relaciones complejas en texto.
- TensorFlow es ideal para tareas avanzadas como generación de texto y traducción automática.

2.2. Scikit-learn: Modelos Clásicos para Minería de Texto

¿Qué es Scikit-learn?

Scikit-learn es una biblioteca de Machine Learning diseñada para modelos tradicionales, como regresión, clasificación y clustering. A diferencia de TensorFlow, que es más adecuado para modelos complejos, Scikit-learn es ideal para tareas de minería de texto con algoritmos clásicos.

Ejemplo de Aplicaciones en Minería de Texto:

- Clasificación de documentos (Ejemplo: correos electrónicos spam vs. no spam).
- Análisis de sentimientos (Ejemplo: comentarios positivos o negativos).
- Agrupamiento de noticias (Ejemplo: clústeres de artículos similares).

Ejemplo en Scikit-learn: Clasificación de Opiniones con Naive Bayes

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline

# Datos de entrenamiento
opiniones = ["Me encanta este producto", "Horrible, nunca lo compraría",
             "Excelente calidad y buen precio", "No me gustó, muy malo"]
etiquetas = [1, 0, 1, 0] # 1 = Positivo, 0 = Negativo

# Creación del modelo con TF-IDF y Naive Bayes
modelo_texto = make_pipeline(TfidfVectorizer(), MultinomialNB())
modelo_texto.fit(opiniones, etiquetas)

# Evaluación del modelo
nueva_opinion = ["Es un producto excelente y muy recomendable"]
print("Predicción:", modelo_texto.predict(nueva_opinion))
```

Explicación del Código:

1. Se usa TF-IDF para convertir el texto en una representación numérica.
2. Se entrena un modelo de Naive Bayes para clasificación.
3. Se realiza una predicción sobre una nueva opinión.

Ejemplo de Salida:

```
Predicción: [1] # El modelo clasifica la opinión como positiva.
```

Conclusión: Scikit-learn es más rápido y eficiente para modelos tradicionales de clasificación de texto. Los modelos como Naive Bayes o SVM son altamente interpretables y fáciles de entrenar.

2.3. Comparación entre TensorFlow y Scikit-learn

Característica	TensorFlow	Scikit-learn
Modelos soportados	Redes neuronales profundas	Algoritmos clásicos (SVM, árboles, regresión)
Facilidad de uso	Curva de aprendizaje más alta	API simple y rápida
Escalabilidad	Soporta GPU y TPU para modelos grandes	Limitado a CPU y memoria local
Preprocesamiento	TensorFlow Data API	Pandas y sklearn.preprocessing
Ejemplo de aplicación	Chatbots, traducción automática	Clasificación de texto, análisis de sentimientos

Conclusión General:

- Scikit-learn es una opción rápida y eficiente para modelos tradicionales de minería de texto.
- TensorFlow es más potente para modelos avanzados como redes neuronales recurrentes (RNN) o transformers.
- La elección entre TensorFlow y Scikit-learn dependerá del problema a resolver y la complejidad del modelo requerido.

Sección 3: Ejercicio Integrador y Publicación en el Foro

Objetivo de la Sección: En este ejercicio integrador, aplicaremos todo lo aprendido sobre Minería de Texto, desde el preprocesamiento de datos hasta la implementación de modelos de Machine Learning con Scikit-learn y TensorFlow. Finalmente, compartiremos los resultados en el foro, fomentando la discusión y el aprendizaje colaborativo.

3.1. Ejercicio Integrador: Análisis de Opiniones en Redes Sociales

Contexto: Una empresa quiere analizar opiniones de clientes en redes sociales para determinar si son positivas o negativas. Para ello, necesita un modelo de Machine Learning que clasifique automáticamente los comentarios en dos categorías: **1 (Positivo)** / **0 (Negativo)**.

Flujo de Trabajo del Ejercicio:

1. Preprocesar un conjunto de opiniones en español.
2. Aplicar TF-IDF para convertir el texto en características numéricas.
3. Entrenar un modelo de clasificación con Scikit-learn.
4. Comparar resultados con una red neuronal en TensorFlow.

Paso 1: Preparación de los Datos

```
import pandas as pd
import numpy as np
import nltk
import string
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

# Descargar recursos necesarios de NLTK
nltk.download("punkt")
nltk.download("stopwords")

# Datos de ejemplo (opiniones de clientes)
opiniones = [
    "Este producto es excelente, me encantó",
    "Horrible, nunca lo compraría",
    "Muy bueno, lo recomiendo totalmente",
    "No me gustó, la calidad es mala",
    "Una compra increíble, vale la pena",
    "Muy malo, pésima experiencia",
]

# Etiquetas: 1 = Positivo, 0 = Negativo
etiquetas = np.array([1, 0, 1, 0, 1, 0])

# Función de preprocesamiento del texto
def limpiar_texto(texto):
    texto = texto.lower().translate(str.maketrans("", "", string.punctuation))
    tokens = word_tokenize(texto)
    tokens_limpios = [word for word in tokens if word not in stopwords.words("spanish")]
    return " ".join(tokens_limpios)

# Aplicar preprocesamiento
opiniones_limpias = [limpiar_texto(op) for op in opiniones]
```

Explicación del Código:

- Se convierten las opiniones a minúsculas y se eliminan signos de puntuación.
- Se realiza tokenización (separación en palabras).
- Se eliminan palabras vacías (stopwords).

Paso 2: Clasificación con Scikit-learn (Naive Bayes)

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Dividir en conjuntos de entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(opiniones_limpias, etiquetas,
                                                    test_size=0.2, random_state=42)

# Vectorización TF-IDF + Naive Bayes
modelo_nb = make_pipeline(TfidfVectorizer(), MultinomialNB())
modelo_nb.fit(X_train, y_train)

# Evaluar modelo
y_pred_nb = modelo_nb.predict(X_test)
print("Precisión Naive Bayes:", accuracy_score(y_test, y_pred_nb))
print("\nReporte de Clasificación:\n", classification_report(y_test, y_pred_nb))
```

Explicación del Código:

- TF-IDF convierte el texto en una representación numérica.
- Naive Bayes es un modelo eficiente para clasificación de texto.
- Se evalúa la precisión del modelo con un conjunto de prueba.

Paso 3: Clasificación con TensorFlow (Red Neuronal)

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Embedding, Dense, LSTM
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Tokenización del texto con TensorFlow
tokenizer = Tokenizer(num_words=1000, oov_token="<OOV>")
tokenizer.fit_on_texts(opiniones_limpias)
sequences = tokenizer.texts_to_sequences(opiniones_limpias)
padded_sequences = pad_sequences(sequences, maxlen=10, padding='post')

# Dividir en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(padded_sequences, etiquetas,
                                                    test_size=0.2, random_state=42)

# Definir el modelo de red neuronal
modelo_tf = keras.Sequential([
    Embedding(input_dim=1000, output_dim=16, input_length=10),
    LSTM(16),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid')
])

# Compilar y entrenar el modelo
modelo_tf.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
modelo_tf.fit(X_train, y_train, epochs=5, verbose=1)

# Evaluar modelo
y_pred_tf = (modelo_tf.predict(X_test) > 0.5).astype("int32")
print("Precisión Red Neuronal:", accuracy_score(y_test, y_pred_tf))
```

Explicación del Código:

- Tokenización y padding con TensorFlow para convertir texto en secuencias numéricas.
- Red LSTM para aprender patrones en secuencias de texto.
- Evaluación del modelo con precisión.

Instrucciones para compartir resultados en el Foro

Publica un mensaje en el foro de la clase con los siguientes puntos:

- Resumen de los modelos aplicados y los resultados obtenidos.
- Capturas o gráficos de la precisión de Naive Bayes y la Red Neuronal.
- Comparación entre ambos modelos (ventajas y desventajas).
- ¿Cuál crees que es mejor para este problema? ¿Por qué?

Ejemplo de Publicación en el Foro

Análisis de Opiniones en Redes Sociales

Resumen del Análisis: "Entrené dos modelos para clasificar opiniones de clientes: un modelo de Naive Bayes con Scikit-learn y una Red Neuronal con TensorFlow. La precisión obtenida fue la siguiente:"

- Naive Bayes: 78% de precisión
- Red Neuronal (LSTM): 85% de precisión

Comparación entre modelos: "La red neuronal tiene mejor desempeño, pero requiere más datos y mayor tiempo de entrenamiento. En cambio, Naive Bayes es más rápido y fácil de interpretar."

Discusión en el foro:

- ¿Crees que una red neuronal siempre será mejor que Naive Bayes?
- ¿Cómo podríamos mejorar estos modelos con más datos?

- ¿Qué otras técnicas podríamos probar?

Bibliografía

1. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
2. Jurafsky, D., & Martin, J. H. (2021). *Speech and Language Processing (3rd Edition)*. Pearson.
3. Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques*. Elsevier.
4. Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)*. O'Reilly.
5. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
6. TensorFlow Developers. (n.d.). *TensorFlow: An Open Source Machine Learning Framework*. Recuperado de <https://www.tensorflow.org/>
7. Scikit-learn Developers. (n.d.). *Scikit-learn: Machine Learning in Python*. Recuperado de <https://scikit-learn.org/>
8. Hugging Face Developers. (n.d.). *Transformers Library*. Recuperado de <https://huggingface.co/transformers/>
9. Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and Statistical Modeling with Python. *SciPy Conference*.